CodiNeo: A Documentation Generation System Using Hash-Based Virtual Structural Copy Algorithm for Design Pattern Learning in DEVCON Luzon

by

Balbarosa, John Andrew
De Leon, Miryl Z.
Santander, Josephine J.

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Computer Science with Specialization in Software Engineering

at

FEU Institute of Technology

March/2026

Ms. Kim Giselle Bautista
Thesis Adviser

APPROVAL AND ACCEPTANCE SHEET

The thesis entitled **"CodiNeo: A Documentation Generation System Using Hash-Based Virtual Structural Copy Algorithm for Design Pattern Learning in DEVCON Luzon"** prepared and submitted by:

Balbarosa, John Andrew
De Leon, Miryl Z.
Santander, Josephine J.

In partial fulfillment of the course of requirement for the Degree of Bachelor of Science in Computer Science with specialization in Software Engineering has been examined and is hereby recommended for approval.

MS. MAY FLORENCE SAN PABLO
Panelist 1

DR. DENNIS B. GONZALES
Panelist 2

DR. HADJI TEJUCO
Head Panelist

Accepted as partial fulfillment of the requirements for the **Degree of Bachelor of Science in Computer Science with specialization in Software Engineering**.

MS. KIM GISELLE BAUTISTA
Thesis Adviser

MS. ELISA MALASAGA
Course Adviser

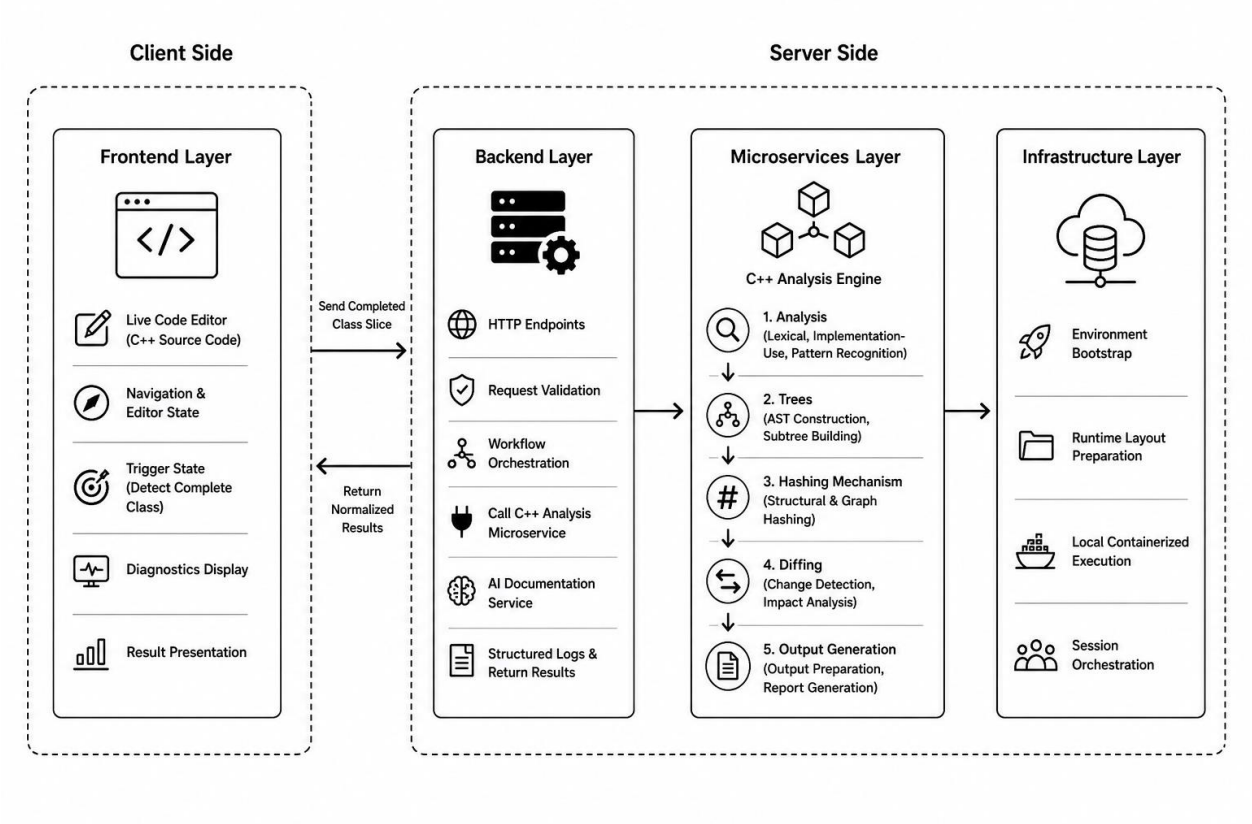
DR. SHANETH C. AMBAT
Department Head

Date

ACKNOWLEDGMENT(OPTIONAL)

The student may choose to include an acknowledgment page to thank those whom they wish to show gratitude to such as their parents, advisers etc. The content must be in English.

TABLE OF CONTENTS



List of Equations

List of Figures

List of Tables

Chapter 1

INTRODUCTION

Modern software development requires more than producing code that functions correctly. In professional and collaborative environments, developers are also expected to understand, maintain, explain, and extend existing codebases. This makes code comprehension an important part of software development, especially when contributors need to study the structure and purpose of a system before modifying or extending it. For interns, novice developers, and new contributors, this process can be challenging because they are often required to understand code written by others while still developing their own technical skills (Rukmono et al., 2021; Romeo et al., 2024).

One important concept that supports code understanding is the use of software design patterns. Design patterns provide reusable solutions, common structures, and shared vocabulary for object-oriented software development. They help developers understand how classes, methods, responsibilities, and relationships are organized within a system. In professional software development, knowledge of design patterns is valuable because it supports code review, system maintenance, communication of design decisions, and understanding of existing software structures (Nesteruk, 2022).

However, design patterns can be difficult for learners to understand when they are presented only as theories, diagrams, or simplified examples. Interns and novice developers may know the definition of patterns such as Singleton, Factory, Builder, Adapter, Strategy, or Repository, but they may still struggle to recognize how these patterns appear in actual C++ source code. In real implementation, design-pattern evidence may be found across class declarations, methods,

attributes, object creation logic, and relationships between components. This creates a gap between learning design patterns conceptually and identifying them in practical source-code structures.

Documentation can help bridge this gap by explaining the purpose, behavior, and organization of software components. Clear documentation can guide learners in understanding what a class does, why certain structures were used, and how a design concept is reflected in the implementation. However, documentation is often produced manually, updated inconsistently, or created only after development has already progressed. When implementation and documentation become disconnected, developers may experience greater difficulty understanding the actual structure of a system (Romeo et al., 2024).

Because of this, there is a need for a support system that can help learners study design patterns directly from source code. Such a system should not only identify possible design-pattern structures but also present the evidence behind the detection and generate documentation-oriented explanations that are understandable to learners. Static and structural code analysis can support this need by inspecting source-code elements, relationships, and structures without relying only on plain-text interpretation, although existing analysis tools may still involve trade-offs in precision, performance, scalability, and developer interpretation of outputs (Park et al., 2021; Nguyen Quang Do et al., 2020).

This study proposes CodiNeo, a design-pattern learning support system for C++ programming. The system provides learning modules that introduce software design-pattern concepts and a C++ analysis studio that allows users to examine actual source code, detect

supported design-pattern evidence, view relevant structural elements, and generate documentation-oriented outputs. Through this approach, CodiNeo aims to help learners bridge the gap between understanding design patterns in theory and recognizing them in actual C++ implementation.

The study is grounded in structure-aware source-code analysis. Instead of treating code as plain text alone, the system examines class-level structures, methods, attributes, and relationships that may serve as evidence for supported design patterns. Abstract Syntax Tree and graph-based representations are relevant to this approach because they allow source code to be analyzed through structural elements and relationships among program components (Fan et al., 2021; TehraniJamsaz et al., 2023; Zhang & Saber, 2025). This direction is aligned with the current study's focus on using C++ class-level analysis, design-pattern evidence detection, and documentation-oriented outputs to support code understanding, and design-pattern learning.

1.1 Background of the Study

Software development has become increasingly collaborative, requiring contributors to work with existing codebases, understand system structures, and communicate design decisions with other members of a team. In this setting, the ability to understand code is just as important as the ability to write code. Developers often need to read and interpret existing software before they can safely modify, extend, or document it. For new contributors, especially interns and novice developers, this process may become difficult because they are still adjusting to actual development practices, unfamiliar technologies, and project-based workflows (Rukmono et al., 2021).

This concern is relevant in learning-oriented and community-based environments such as DEVCON Luzon. DEVCON Luzon operates as part of a non-profit technology community that provides Filipino technology enthusiasts with opportunities for support, learning, and collaboration. Its chapter-based environment allows volunteers, interns, and contributors to participate in projects, bootcamps, and development activities that connect technical learning with practical work. In this context, learning and collaboration become important parts of the development process because project work may involve contributors with different levels of experience and different technical roles.

DEVCON Luzon's internship program reflects this learning-oriented environment. Interns are usually recruited from current or previous chapter volunteers and undergo an interview process before joining the program. The internship is intended to provide volunteers with real-world exposure to work, allowing them to form teams, choose community-based projects, and learn the skills needed to execute their tasks effectively. Bootcamps are also integrated into the internship program, where topics are designed based on identified skills or skill gaps and are then applied directly to project work.

However, this setup also requires interns and novice developers to adapt quickly to unfamiliar tools, technologies, and development practices. One of the identified challenges in DEVCON Luzon's professional development projects and internship program is the unfamiliarity of team members with certain technology stacks. This shows that the problem is not limited to

programming ability alone. It also involves understanding how existing software systems are built and how technical concepts are applied in actual development.

One important part of this transition is design-pattern understanding. As interns and novice developers prepare for professional software development environments, they need to learn not only programming syntax but also how software systems are structured and designed. Design patterns are useful in this preparation because they describe recurring structures, responsibilities, and interactions in object-oriented software. However, learners may still struggle to recognize how these concepts appear in real source code, especially when the evidence is distributed across class structures, methods, attributes, and relationships (Nesteruk, 2022).

In internship and bootcamp environments, learners often depend on mentors, reviewers, or senior developers to explain how source code works and why certain design patterns may be present. While this guidance is valuable, it can be time-consuming and may not always be available for every code sample. Manual documentation can also support learning, but it may be incomplete, delayed, or inconsistent with the actual implementation. Prior research on design, implementation, and documentation drift shows that when documentation no longer reflects the actual system, software comprehension becomes more difficult (Romeo et al., 2024).

To address this need, this study focuses on the development of CodiNeo, a C++ design-pattern learning and documentation support system for DEVCON Luzon. The system includes learning modules that introduce users to selected software design patterns and a C++ analysis studio that allows users to apply these concepts to actual source-code examples. Through the

studio, users can submit C++ code, detect supported design-pattern evidence, view highlighted structures, and generate documentation-oriented explanations. In this context, design-pattern evidence refers to structural indications in the source code that help explain how a class or component is organized. AST-based and graph-based representations support this approach because they allow source code to be modeled through syntactic structures and relationships among program elements (Fan et al., 2021; Wang et al., 2021; TehraniJamsaz et al., 2023).

The proposed system serves as a code-understanding and documentation support tool. By generating documentation-oriented outputs and explanations from detected structural evidence, CodiNeo aims to help interns and new contributors interpret C++ code structure, connect design-pattern theory with actual implementation, and understand existing software more effectively. Through this approach, the study aims to bridge the gap between software design theory and practical code understanding. By providing evidence-based C++ code analysis and documentation support, CodiNeo may help interns adapt more effectively to unfamiliar codebases, improve their understanding of design patterns, and become more prepared for professional software development environment.

1.2 Significance of the Study

This study is significant because it introduces a C++ design-pattern learning support system that uses structure-aware code analysis and documentation-oriented outputs that helps bridge the gap between software design theory and practical code understanding. In collaborative and learning-oriented development environments, interns and novice developers may be able to write functional code but still struggle to understand how source code is organized, how design concepts

appear in actual implementation, and how code should be documented. By using source-code structure and design-pattern evidence as a basis for documentation generation, the proposed system supports learning, and code comprehension.

For interns and novice developers, this study provides a guided way to understand C++ source code. Instead of relying only on delayed explanations from mentors or manual review, They can first study design-pattern concepts through learning modules, then apply those concepts by analyzing actual C++ code. This can help interns better understand how existing software is built and how design ideas are reflected in actual code.

For developers, the system provides a direct workspace for analyzing C++ source code, identifying supported design-pattern evidence, and generating documentation-oriented outputs that can assist in code review, explanation, and documentation preparation.

For mentors and senior developers, this study may help reduce repetitive explanation during onboarding and code review. Mentors remain essential in guiding interns and evaluating project work, but the proposed system can provide an additional support layer by generating initial documentation, diagnostics, and explanations from the submitted code. This allows mentors to focus more on higher-level guidance, project direction, and deeper software design discussions.

For DEVCON Luzon, this study is significant because it addresses a need identified in its internship and bootcamp environment. DEVCON Luzon indicated that interns and team members may experience difficulty adapting to unfamiliar tools and technology stacks. The proposed system

supports this environment by helping interns understand existing software, connect theory with application, and generate clearer documentation that can assist onboarding, project learning, and design-pattern understanding.

For future researchers, this study may serve as a reference for developing tools that combine deterministic source-code analysis with AI-assisted explanation. The system demonstrates how Abstract Syntax Tree and graph-based analysis can be used not only for code inspection, but also as a foundation for documentation generation and learning support. This may encourage further studies on source-code understanding tools, design-pattern learning systems, and documentation support for novice developers.

For the field of software engineering, this study contributes to the development of tools that support code comprehension, documentation, and learning within the development process. Rather than focusing only on error detection or post-development review, the study highlights the value of using structural code analysis to generate explanations that help developers understand how software is built.

Overall, this study supports collaborative software development by helping new contributors understand and document source code more effectively. Through C++ class-level analysis, design-pattern evidence detection, and AI-assisted documentation generation, the proposed system aims to improve internship onboarding, design-pattern learning, and practical code understanding in community-based development environments.

1.3 Statement of the Problem

This study aims to determine how a design-pattern learning support system can analyze C++ source-code structure, detect design-pattern evidence, and generate documentation-

oriented outputs to support code understanding, and design-pattern learning in DEVCON Luzon.

Specifically, this study seeks to address the following problems:

1. How can the proposed system provide learning support for users in understanding software design-pattern concepts?
2. How can the proposed system analyze C++ source code to identify supported design-pattern evidence?
3. How can the proposed system present detected design-pattern evidence in a way that helps users understand the structure and important parts of the analyzed source code?
4. How can documentation-oriented outputs help users connect design-pattern concepts with actual C++ source-code implementation?
5. How useful is the proposed system in supporting design-pattern learning and code understanding in DEVCON Luzon?

1.4 Objectives of the Study

The general objective of this study is to develop a design-pattern learning system that combines learning modules, C++ source-code analysis, design-pattern evidence detection, and documentation-oriented outputs to support code understanding and design-pattern learning in DEVCON Luzon. Specifically, this study seeks to address the following objectives:

1. Develop learning modules that support users in understanding selected software design-pattern concepts.
2. Develop a C++ source-code analysis feature that identifies supported design-pattern evidence from class-level source-code structure.

3. Present detected design-pattern evidence through clear structural outputs that help users understand the important parts of the analyzed source code.
4. Generate documentation-oriented outputs that help users connect design-pattern concepts with actual C++ source-code implementation.
5. Evaluate the usefulness of the proposed system in supporting design-pattern learning and code understanding in DEVCON Luzon.

1.5 Theoretical Framework

The theoretical framework of this study is grounded in software engineering and computer science concepts that explain how source-code structure can be represented, analyzed, interpreted, and used to support documentation generation, code understanding, internship onboarding, and design-pattern learning. These concepts include software documentation and knowledge transfer, internship onboarding and learning support, design patterns, Abstract Syntax Tree representation, graph-based structural representation, static and structural code analysis, deterministic algorithm design, and AI-assisted explanation generation. Together, these foundations support the development of a graph-based and AST-centered documentation generation system for C++ source code.

Software documentation serves as an important foundation of this study because it supports communication, maintenance, and knowledge transfer in software development. Documentation helps explain the purpose, structure, behavior, and responsibilities of software components. For new contributors, especially interns and novice developers, documentation provides guidance in understanding existing systems and reduces the difficulty of studying unfamiliar code. In this study, documentation is treated not only as a written output but also as a learning support mechanism that can help users understand how source code is organized (Romeo et al., 2024).

Internship onboarding and learning support also serve as important foundations of the study. In internship-based development environments, learners are expected to apply programming concepts in actual project work while adapting to unfamiliar tools, technologies, and codebases. This creates a need for support systems that help learners understand software implementation more clearly. In this study, onboarding is viewed as the process of helping new contributors understand the structure and purpose of existing code so they can participate more effectively in development activities (Rukmono et al., 2021).

Design patterns provide the conceptual basis for understanding recurring software structures. They help developers describe common solutions, responsibilities, and interactions within software systems. However, design patterns may be difficult for interns and novice developers to recognize when they are only discussed in theory. In this study, design patterns are used as an interpretive framework for explaining source-code structure. The system analyzes class-level C++ code and identifies evidence that may correspond to supported design-pattern definitions. This evidence then becomes part of the basis for documentation and explanation generation (Nesteruk, 2022).

Abstract Syntax Tree representation provides the structural foundation for analyzing source code. Instead of treating source code as plain text, an AST represents code as a hierarchy of syntactic elements such as declarations, statements, expressions, classes, methods, and relationships among program components. This representation allows the system to examine the organization of code more meaningfully. In this study, AST representation supports the identification of complete C++ class or struct declarations and provides the basis for extracting structural elements that are relevant to documentation and design-pattern learning (Fan et al., 2021).

Graph-based structural representation extends the AST-based view by modeling code elements as interconnected nodes and relationships. Through graph-based representation, the system can examine how class-level elements are connected and how these relationships may provide evidence of design-pattern structures. This is important because design patterns are often reflected not only in individual code elements, but also in the relationships and responsibilities among those elements. Graph-based modeling therefore supports a deeper analysis of source-code structure beyond surface-level text inspection (TehraniJamsaz et al., 2023; Liu et al., 2023).

Static and structural code analysis provide the method for examining source code without executing the program. Static analysis allows the system to inspect declarations, relationships, and structural features during development. Structural code analysis is especially relevant to this study because the proposed system focuses on understanding how code is organized rather than checking runtime behavior. Through structural analysis, the system can extract class-level evidence that may be used to generate documentation tags, diagnostics, unit-test targets, structured reports, and learning explanations (Park et al., 2021; Nguyen Quang Do et al., 2020).

Deterministic algorithm design supports the reliability and repeatability of the system's analysis process. A deterministic approach ensures that the same source-code input produces the same analysis result under the same conditions. This is important because the documentation and explanations generated by the system depend on the structural evidence extracted from the code. By using deterministic analysis before AI-assisted generation, the system provides a stable and explainable foundation for its outputs.

AI-assisted explanation generation serves as the narrative support layer of the proposed system. While deterministic analysis identifies the source-code structure and design-pattern

evidence, the AI component helps transform these structured results into readable explanations. In this study, AI is not treated as the primary analyzer of source-code structure. Instead, it is used to generate documentation-style explanations based on the evidence produced by the system. This separation allows the core analysis to remain deterministic while making the final output more understandable and useful for interns and novice developers. This direction is consistent with studies showing both the usefulness of LLMs in software engineering and the need to ground AI-assisted reasoning in structured information (Hou et al., 2024; Chung et al., 2025).

Together, these theoretical foundations support the development of a system that bridges the gap between software design theory and practical code understanding. By combining documentation principles, onboarding support, design-pattern interpretation, AST and graph-based representation, structural code analysis, deterministic algorithm design, and AI-assisted explanation generation, the study establishes a framework for supporting documentation generation and design-pattern learning in internship-based and collaborative development environments.

1.6 Conceptual Framework

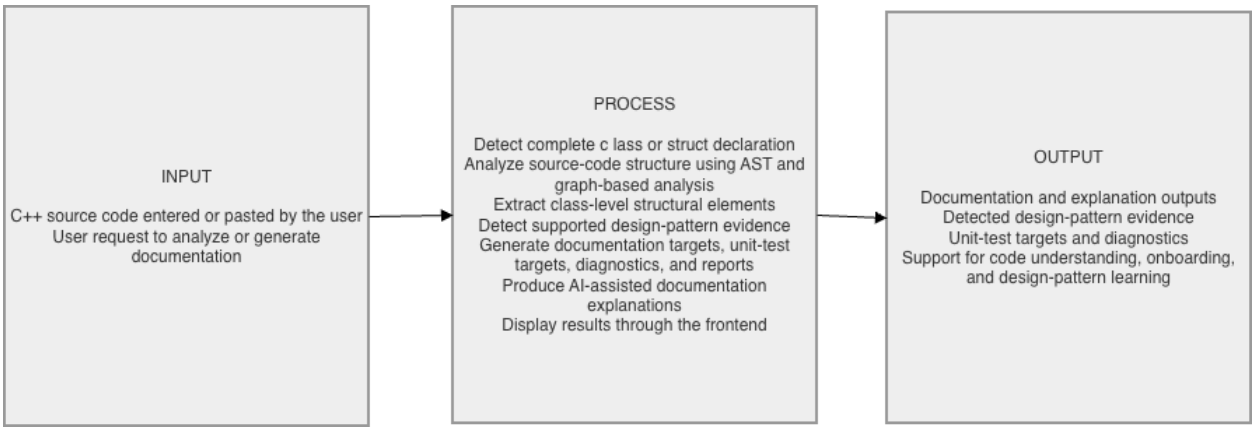


Figure 1. Conceptual Framework

The conceptual framework of this study follows an Input-Process-Output (IPO) model. The framework illustrates how the proposed design-pattern learning support system processes user learning needs and C++ source-code input to produce outputs that support design-pattern learning and code understanding in DEVCON Luzon.

The input component consists of the user's selected pathway, design-pattern learning modules, supported design-pattern catalog entries, and C++ source code entered or pasted by the user. The user may access the system through a learner pathway, where design-pattern concepts are introduced through learning modules, or through a developer/studio pathway, where C++ source code is directly analyzed. The supported design-pattern catalog entries serve as references for identifying possible design-pattern evidence from the analyzed source-code structure.

The process component is divided into two connected parts: the learning support process and the C++ analysis and documentation process. In the learning support process, the system presents selected software design-pattern concepts through learning modules. These modules help users

understand the purpose, structure, and basic implementation ideas of supported design patterns before applying them to actual source-code analysis.

In the C++ analysis and documentation process, the user enters or pastes C++ source code through the frontend interface. The system focuses on complete class or struct declarations because the analysis is class-level and design-pattern-oriented. The frontend sends the submitted code to the backend for validation and orchestration. The backend then forwards the validated input to the C++ analysis microservice.

The C++ analysis microservice performs the core structural analysis of the system. It examines class-level structural elements such as declarations, methods, attributes, relationships, and implementation-use evidence. It conducts lexical and structural analysis, constructs class-level representations, detects supported design-pattern evidence, applies hashing and identity propagation where applicable, and prepares the analysis artifacts. From the detected structure and pattern evidence, the system identifies relevant documentation targets, diagnostics, and possible unit-test targets.

After the analysis artifacts are prepared, the backend organizes the results into documentation-oriented outputs. These outputs are based on the detected code units, design-pattern evidence, documentation targets, diagnostics, and relevant structural information. If AI-assisted explanation generation is configured, the system may use the structured analysis results to generate readable documentation-style explanations. If AI assistance is not configured, the system still presents deterministic documentation-oriented outputs based on the analysis results.

The output component consists of design-pattern learning support, detected design-pattern evidence, highlighted structural elements, diagnostics, documentation targets, possible unit-test targets, structured analysis reports, and documentation-oriented explanations. These outputs are intended to help learners, interns, novice developers, and developers understand software design-pattern concepts, recognize how supported patterns appear in actual C++ source code, and connect theoretical design-pattern learning with practical implementation.

Through this process, the system supports the progression from conceptual learning to practical code understanding. Users can first study selected design-pattern concepts through the learning modules and then apply that knowledge by analyzing C++ source code in the studio. As a result, the conceptual framework shows how CodiNeo supports design-pattern learning, code understanding, and documentation-oriented explanation in the DEVCON Luzon learning environment.

1.7 Scope and Delimitations

1.7.1 Scope

This study focuses on the design, development, and evaluation of CodiNeo, a design-pattern learning support system for C++ programming. The system is intended to support design-pattern learning and code understanding by combining learning modules, C++ source-code analysis, design-pattern evidence detection, and documentation-oriented outputs.

The system includes a learner pathway that provides learning modules for selected software design-pattern concepts. These modules are designed to introduce users to the purpose, structure, and basic implementation ideas of supported design patterns. Through this pathway, users can study design-pattern concepts before applying them to actual C++ source-code analysis.

The system also includes a developer or studio pathway where users can enter or paste C++ source code through a frontend interface. This pathway is intended for users who want to directly analyze C++ code, identify supported design-pattern evidence, and generate documentation-oriented outputs based on the analyzed structure.

The C++ analysis feature focuses on complete class or struct declarations because the system's analysis is class-level and design-pattern-oriented. The system examines relevant structural elements such as declarations, methods, attributes, relationships, and implementation-use evidence that may help explain how the code is organized and how supported design patterns may appear in the submitted source code.

The design-pattern detection feature is limited to the supported design-pattern definitions included in the implemented prototype. These supported patterns serve as references for identifying possible structural evidence in the analyzed C++ code. The system does not claim to detect all existing design patterns, but focuses only on the design-pattern cases covered by its implemented detection logic.

The system generates outputs based on the analyzed source-code structure and detected design-pattern evidence. These outputs may include detected pattern evidence, highlighted structural elements, documentation targets, diagnostics, structured analysis reports, documentation-oriented explanations, and possible unit-test targets for supported cases. These outputs are intended to help users understand the important parts of the analyzed code and connect design-pattern concepts with actual C++ implementation.

The study is contextualized within DEVCON Luzon as a collaborative and learning-oriented development environment. The system is intended to support learners, interns, novice developers, and developers who need assistance in understanding design-pattern concepts, recognizing supported design-pattern evidence in C++ code, and generating documentation-oriented explanations that support code understanding.

The evaluation of the system focuses on its usefulness in supporting design-pattern learning and code understanding in DEVCON Luzon. It also considers the system's usability, clarity of outputs, functional suitability, and ability to provide relevant learning and documentation-oriented support based on selected C++ code samples and supported design-pattern cases.

1.7.2 Delimitations

This study is limited to C++ source code and to the subset of C++ class or struct declarations supported by the implemented prototype. It does not cover all programming languages, all C++ constructs, or all possible object-oriented programming structures.

The learning module component is limited to selected software design-pattern concepts included in the prototype. It is not intended to replace a complete software engineering course, programming course, or full design-pattern curriculum. The learning modules serve only as supplementary learning support for users who need introductory guidance before applying the concepts to C++ code analysis.

The system assumes that the submitted C++ source code is sufficiently complete and syntactically valid for class-level structural analysis. Code with incomplete declarations, unsupported syntax, heavily macro-based structures, advanced template metaprogramming, generated code, or inaccessible external dependencies may not be fully analyzed by the prototype.

The design-pattern detection feature is limited to the supported design-pattern definitions implemented in the system. The system does not guarantee complete or universal design-pattern recognition across arbitrary codebases. Detected design-pattern evidence should be treated as learning and documentation support, not as a final judgment of design correctness or code quality.

The core design-pattern detection process is based on static and structure-based analysis. Any testing or validation feature included in the prototype is limited to supported templates or selected design-pattern cases and is not intended to fully verify the functional correctness, runtime behavior, performance, memory safety, or security of arbitrary user programs.

If AI-assisted explanation generation is used, it is limited to generating documentation-style explanations from the structured analysis results provided by the system. The AI component does not replace the deterministic structural analysis performed by the system and should not be treated as the sole basis for evaluating code structure, design quality, or pattern correctness.

The evaluation of the system is conducted using selected users, selected C++ code samples, supported design-pattern cases, and controlled testing scenarios. Therefore, the findings are limited to the prototype's implemented features, supported code structures, selected evaluation participants, and the DEVCON Luzon learning and development context.

1.8 Definition of Terms

Abstract Syntax Tree (AST) – A hierarchical tree representation of the syntactic structure of source code, where each node corresponds to a construct defined by the programming language grammar.

Algorithmic Complexity – Refers to the measurement of computational resources required by an algorithm, typically expressed in terms of time and space as a function of input size. In this study, it emphasizes the scalability constraints of structural comparison and traversal operations.

Dependency Graph – A graph representation where nodes represent entities (e.g., classes, functions, modules) and edges represent relationships or dependencies among them. It serves as the structural model used in this research.

Graph-Based Representation – A model that represents program elements as nodes and their relationships as edges, allowing multiple dependencies and interactions to be analyzed simultaneously.

Graph Memory Copy Algorithm – The proposed algorithm in this study performs localized graph traversal and selective node duplication to detect and correct structural non-conformance efficiently.

Graph Traversal – The process of visiting and analyzing nodes and edges within a graph according to a defined strategy to extract structural or semantic information.

Heuristic-Based Approach – An analytical method that employs rule-of-thumb strategies or localized decision-making to efficiently explore large problem spaces without exhaustive computation.

Language-Agnostic – A characteristic of a system or algorithm that allows it to operate across multiple programming languages without dependence on language-specific rules or constructs.

Manual Code Review – The process in which developers or reviewers inspect source code by hand to identify errors, inconsistencies, or violations of coding standards.

Structural Consistency – The condition where relationships among components adhere to predefined architectural or design constraints.

Structural Inconsistency – A deviation in code organization or architecture that violates established structural rules or design standards while possibly preserving program behavior.

Structural Validation – The systematic verification of relationships and dependencies within a codebase to ensure adherence to design rules.

Syntax-Centric Analysis – An analysis approach that focuses primarily on the syntactic form of source code rather than its semantic meaning or structural relationships.

System-Level Validation – Evaluation of a system's correctness, efficiency, and behavior under realistic operational conditions.

Chapter 2

REVIEW OF RELATED LITERATURES AND STUDIES

2.1 Code Understanding in Collaborative and Learning-Oriented Development

Modern software development is often performed by teams composed of individuals with different levels of experience, roles, and technical backgrounds. In collaborative projects, developers are not only expected to write functional code but also to understand existing systems, communicate design decisions, and maintain code that can be extended by others. This makes program comprehension an important activity in software engineering because developers must first understand the structure and purpose of existing code before they can safely modify or extend it.

Program comprehension becomes more challenging when new contributors are introduced to unfamiliar software systems. Interns, novice developers, and new team members may have basic programming knowledge but may still struggle to understand how a project is organized, how software components interact, and how design decisions are reflected in implementation. In this context, the difficulty is not limited to writing new code. It also includes understanding code written by others, identifying important structures, and learning how the codebase follows certain design practices.

Studies on software maintenance and program comprehension show that developers often spend a significant amount of time understanding existing code before making changes. When a system lacks clear organization or sufficient explanation, the effort required to interpret the code increases. Rukmono et al. (2021) emphasize that explaining software architecture to less experienced developers is an important concern

because the way a system is presented affects how new learners understand it. Similarly, research on architecture and documentation drift shows that when implementation and documentation become disconnected, developers may have greater difficulty understanding the actual structure of a system (Romeo et al., 2024).

In learning-oriented environments, this issue becomes more important because new contributors are still developing both technical skills and conceptual understanding. They may be learning unfamiliar tools, frameworks, programming languages, or development practices while also contributing to actual projects. This creates a need for support mechanisms that help learners interpret code structure more clearly and connect implementation details with software design concepts.

In the context of this study, code understanding is treated as a central requirement for internship onboarding and design-pattern learning. A support system that can analyze source-code structure and present understandable outputs may help interns and novice developers study unfamiliar code more effectively. This provides the foundation for developing a documentation-oriented system that supports learning and code comprehension in collaborative development environments.

2.2 Software Documentation and Knowledge Transfer

2.3

2.4 Limitations of Existing Static and AST-Based Solutions

As software systems continue to grow in complexity, researchers and practitioners have introduced a variety of automated tools to assist developers in maintaining code quality and structural consistency. These tools are designed to reduce human error, accelerate development processes, and help developers identify potential issues within large software systems. Among the most widely adopted approaches in modern development environments are AI-driven coding assistants, static analysis tools, and graph-based structural analysis techniques (Krishnan, 2025). While these technologies provide valuable support for software development, existing research indicates that each approach presents important limitations when applied to large collaborative repositories (Soliman et al., 2021; Soto et al., 2020).

2.4.1 Limitations of AI-Driven Coding Assistants

One of the most significant developments in recent years is the rise of AI-driven coding assistants powered by large language models (LLMs). These systems are capable of

generating code, suggesting improvements, summarizing programs, and assisting developers with debugging tasks. Studies on large language models in software engineering show that these tools are increasingly integrated into development workflows to support programming activities such as automated documentation, code explanation, and code generation (Hou et al., 2024). For many developers, particularly those with less experience, AI-assisted tools provide immediate guidance when implementing programming tasks.

The capabilities of LLM-based systems have improved considerably over time. Empirical evaluations show that modern language models are capable of producing syntactically correct code across multiple programming languages. Chen et al. (2021) demonstrated that models such as Codex can generate functional code for a wide range of programming tasks and assist developers in solving common programming problems. As a result, AI-based assistants are often considered practical tools for improving development productivity and supporting junior developers during the coding process.

Recent studies on development automation further support that tools integrated within modern software pipelines significantly improve development efficiency by reducing manual intervention, accelerating issue detection and resolution, and enabling continuous feedback throughout the development lifecycle (Pranav et al., 2025). Automation-driven practices within structured development workflows allow teams to streamline repetitive tasks and maintain consistency, thereby contributing to improved developer productivity and faster development cycles.

Despite these advantages, research also highlights several limitations in the use of AI-driven coding systems for maintaining structural consistency in collaborative repositories. Hou et al. (2024) explain that large language models often struggle to generalize across

different software projects and organizational contexts. Because these systems rely on probabilistic predictions derived from training data, the generated code may appear syntactically correct while still failing to follow project-specific coding conventions. In repositories that maintain strict architectural standards, this limitation becomes particularly problematic because generated code may diverge from the structural expectations of the project (AST-T5; K-ASTRO).

A major technical factor contributing to this limitation lies in the computational architecture of transformer-based language models. These models rely on self-attention mechanisms that compare tokens within an input sequence in order to determine contextual relationships. However, this architecture introduces quadratic computational complexity relative to the number of tokens in the input sequence (Keles, Wijewardena, & Hegde, 2022). As the size of the input grows, computational cost increases significantly, making it difficult for language models to process very large contexts efficiently. In large collaborative repositories where thousands of files and functions exist simultaneously, language models may only analyze limited portions of the codebase at a time. Consequently, important architectural relationships across the repository may be overlooked when the system processes only partial segments of the project.

Beyond individual coding assistants, recent studies have also explored the use of AI agents that combine language models with planning mechanisms, memory systems, and external tools. These agents are designed to coordinate multiple development utilities and perform structured workflows across complex programming tasks. According to Krishnan (2025), AI agents can orchestrate development tools and execute multi-step reasoning processes to assist with software engineering activities. Although these systems represent a

promising advancement, they still depend on explicit definitions of project rules and coding standards. Without deterministic representations of architectural guidelines, AI agents continue to rely on probabilistic reasoning and therefore cannot guarantee structural conformance in collaborative repositories.

2.4.2 Limitations of Static Analysis Tools

In contrast to AI-based approaches, static analysis tools provide a deterministic method for evaluating source code. Static analysis techniques examine program structures without executing the program, enabling them to detect potential issues through structural inspection. Many static analysis tools rely on Abstract Syntax Trees (ASTs) to represent the hierarchical structure of source code. By modeling programming constructs as tree structures, AST-based analysis enables tools to identify relationships between functions, variables, and control structures within a program (Alikhanifard et al., 2025; SOLIDIFFY; Zhang & Saber, 2025).

However, static analysis tools also introduce important trade-offs. Park, Lee, and Ryu (2021) explain that static analysis systems must balance performance, precision, and soundness, and improvements in one dimension often reduce performance in another. Highly precise analysis techniques require significant computational resources, while lightweight analysis approaches may fail to detect deeper structural issues. As software repositories increase in size, these trade-offs become increasingly difficult to manage because the analysis must process larger volumes of source code.

The effectiveness of static analysis tools is also influenced by developer behavior. Nguyen Quang Do, Wright, and Ali (2020) found that developers often prioritize warnings that are easy to understand and resolve rather than those ranked as most severe by the

analysis system. When warnings appear overly complex or difficult to interpret, developers may ignore them entirely. In collaborative development environments where junior developers frequently contribute code, this behavior may reduce the practical effectiveness of static analysis tools as mechanisms for maintaining structural consistency (Beck et al., 2021; Motwani & Brun, 2021).

2.4.3 Representation Learning via Graph Neural Networks

Research on program representation and structural code analysis further highlights the importance of maintaining consistent architectural organization within software systems. Structural inconsistencies between program elements can complicate automated analysis tasks, particularly when analysis tools rely on structural representations such as Abstract Syntax Trees (ASTs) or graph-based models to interpret relationships among code components (Wang et al., 2021). Recent studies on structure-aware code models and AST-based analysis further demonstrate that structural representations play a critical role in capturing program relationships and enabling effective automated software analysis (Zhang & Saber, 2025; Gong et al., 2024; Zhang et al., 2025).

Recent advances in program analysis have explored the use of Graph Neural Networks (GNNs) for learning representations of structured program data. GNNs enable representation learning directly on graph structures by iteratively propagating information through graph edges. Through this process, the representation of a node can be influenced by the structural context of its neighboring nodes (Liu et al., 2022).

In program analysis, this capability allows GNNs to infer the semantic role of a code element based on its surrounding structural relationships. Liu et al. (2023) explain that nodes

within graph representations such as Code Property Graphs may derive meaning from their structural context, enabling machine learning systems to identify patterns in control and data dependencies across programs.

However, these learned representations are inherently probabilistic. While graph neural networks can approximate semantic relationships within program structures, the inferred representations remain statistical approximations rather than deterministic guarantees of structural conformance. As a result, GNN-based approaches may still struggle to provide strict architectural validation in collaborative software repositories (AST-T5; K-ASTRO).

2.4.4 Computational Efficiency of AST-Based Graph Representations

Recent research has explored the efficiency trade-offs of different graph-based program representations used in software analysis. Abstract Syntax Trees (ASTs) are widely used because they capture the syntactic structure of programs while remaining relatively simple to construct and analyze (Accurate Abstract Syntax Tree Differencing).

However, several studies have attempted to enhance AST representations by incorporating additional semantic information such as Control Flow Graphs (CFGs) and Data Flow Graphs (DFGs) to better capture program behavior and relationships between operations. These hybrid graph representations aim to provide richer structural context for tasks such as code clone detection, program similarity analysis, and automated reasoning about software structure (A Structural Approach to Program Similarity Analysis; SOLIDIFFY).

Despite their improved semantic expressiveness, these augmented representations also introduce additional structural complexity that may affect computational efficiency when applied to large code repositories.

To better understand the computational implications of these design choices, Zhang and Saber (2025) conducted an empirical evaluation comparing multiple AST-based graph representations. Their study systematically measured graph generation cost, storage requirements, graph density, and inference time across several configurations combining syntax trees with control-flow, data-flow, and flow-augmented structures. The results of this evaluation are summarized in table below.

Code Representations	Generation Cost (s)	Graph Storage Cost (MB)	Average Graph Density	Inference Time (s)
AST	14.055	527.54	0.0072	705.680
AST + CFG	14.224	591.00	0.0085	710.026
AST + DFG	305.095	533.38	0.0073	708.282
AST + CFG + DFG	305.407	596.85	0.0087	713.923
AST + FA	16.910	1152.00	0.0202	748.402
AST + FA + CFG	17.371	1215.44	0.0216	752.855
AST + FA + DFG	312.535	1157.83	0.0204	751.140
AST + FA + CFG + DFG	313.585	1221.40	0.0217	756.463

Table 1. Computational Overhead and storage Cost of AST-Based Hybrid Graph Representation (Zhang et. al, 2025)

The findings presented in Table 1 reveal clear differences in computational efficiency between the evaluated representations. The pure AST representation achieved the most efficient performance, requiring only 14.055 seconds for graph generation while consuming approximately 527 MB of storage. In contrast, augmenting the AST with data-flow relationships substantially increased processing overhead.

Hybrid structures that integrate multiple flow relationships further increased both graph density and storage requirements. These results highlight a critical trade-off between structural expressiveness and computational efficiency in graph-based program analysis.

For collaborative development environments that require repeated analysis across large repositories, this trade-off becomes particularly important.

Taken together with prior research on software development tools, these observations reveal a broader limitation in existing approaches to structural code analysis. AI-based coding assistants provide useful support for developers but rely on probabilistic reasoning that cannot guarantee structural conformance. Static analysis tools offer deterministic evaluation but may struggle with scalability and usability in large repositories. Meanwhile, graph-based and GNN-driven methods provide richer structural modeling capabilities yet introduce significant computational costs.

Consequently, existing tools often focus either on assisting developers during coding or analyzing isolated program structures, while relatively few mechanisms are designed specifically to evaluate architectural conformance across large collaborative repositories (Soliman et al., 2021; Soto et al., 2020).

This limitation motivates the exploration of more efficient structural analysis mechanisms capable of combining deterministic evaluation, structural accuracy, and computational efficiency, which forms the basis for the structural differencing approach proposed in this study.

2.5 Algorithmic Frameworks and Structural Analysis

The limitations of existing approaches discussed in the previous section highlight the need for more efficient mechanisms capable of evaluating structural conformance in large collaborative repositories. While AI-driven tools assist developers in generating and modifying code, and static analysis tools provide deterministic inspection of program structures, both approaches encounter practical limitations when applied to large-scale software environments. As repositories expand and development teams grow, maintaining architectural consistency requires algorithmic frameworks capable of analyzing structural relationships efficiently while minimizing computational overhead (Soliman et al., 2021; Soto et al., 2020).

To address this challenge, several theoretical and algorithmic foundations from program analysis, graph theory, formal language processing, and memory-efficient computation can be integrated to support scalable structural conformance analysis. The following subsections describe these foundational concepts and their relevance to the proposed approach.

2.5.1 Deterministic Pushdown Automata (DPDA) in Code Conformance

To ensure accurate structural validation of source code, parsing architectures must operate within the theoretical boundaries of formal language processing. A Deterministic Pushdown Automaton (DPDA) provides a suitable computational model for recognizing

deterministic context-free languages, which include many programming language grammars. A DPDA extends the capabilities of a finite-state machine by incorporating a stack memory structure that enables the system to track nested constructs such as recursive expressions, scope blocks, and hierarchical program elements.

In classical computation theory, an end-marker symbol is appended to the input sequence to allow the automaton to detect the termination of the parsing process and return to an empty stack state. This mechanism ensures that the parser can accurately determine when a complete syntactic structure has been processed. Because deterministic context-free languages represent a subset of non-ambiguous context-free grammars, DPDA-based parsing ensures consistent and unambiguous interpretation of program structures.

Deterministic parsing is particularly important for structural conformance analysis. Parsing systems that rely on nondeterministic transitions often depend on runtime contextual information to resolve ambiguities, which may prevent deterministic evaluation of program structures. Recent research indicates that certain grammar classes, such as LR (1) grammars, can be directly transformed into deterministic pushdown automata, enabling efficient parsing without nondeterministic transitions.

This deterministic parsing behavior provides a reliable foundation for evaluating structural equivalence between program representations. In many program analysis techniques, source code is transformed into Abstract Syntax Trees (ASTs), which represent the syntactic structure of programs as hierarchical tree structures. AST mapping algorithms analyze the correspondence between nodes of different program representations to identify structural changes such as additions, deletions, and updates within the codebase (Fan et al., 2021).

Accurate AST mappings enable tools to represent code evolution through edit actions on tree nodes, which reflect structural modifications in the program's syntax. These mappings play a crucial role in many software engineering tasks, including code change analysis, automated program repair, and mining software evolution patterns (Fan et al., 2021).

2.5.2 Graph Theory and Static Analysis Foundations

Graph theory provides a fundamental framework for modeling software systems as interconnected structures composed of nodes and edges. In program analysis, graph representations allow structural relationships between program components to be analyzed more effectively than purely sequential representations. Nodes typically represent programming constructs such as functions, variables, or expressions, while edges represent structural or dependency relationships between these elements.

One of the most widely used graph representations in program analysis is the Abstract Syntax Tree (AST), which captures the hierarchical syntactic structure of source code. Each node within an AST represents a programming construct, while parent-child relationships represent syntactic nesting within the program. This structure enables analysis systems to inspect program composition and detect structural patterns (Zhang & Saber, 2025; Park, Lee, & Ryu, 2021; Alikhanifard et al., 2025).

However, AST-only analysis may not fully capture the semantic relationships between program components. To address this limitation, researchers have proposed combining AST representations with additional graph structures such as Control Flow Graphs (CFG) and Data Flow Graphs (DFG). These hybrid representations integrate syntactic, control-flow,

and data-flow relationships to provide a more comprehensive representation of program behavior (Liu et al., 2022; Park, Lee, & Ryu, 2021).

Despite their expressive capabilities, hybrid graph models often introduce increased computational complexity and memory overhead when applied to large-scale codebases.

2.5.3 Shadow Trees and Virtual DOM Abstractions

Efficient structural analysis of large program graphs requires mechanisms that minimize unnecessary computation and memory consumption. Resource-aware decision models provide a strategy for achieving this efficiency by focusing analysis on localized portions of the graph rather than performing exhaustive traversal of the entire structure.

Managing structural transformations in complex hierarchical systems requires mechanisms that allow isolated structural updates without disrupting the entire system representation. A conceptual approach inspired by Shadow DOM architectures in web development introduces the concept of shadow trees for structural isolation.

In web systems, the Document Object Model (DOM) represents the structure of a document as a hierarchical tree. However, large applications often experience cascading conflicts when global modifications affect multiple components simultaneously. The Shadow DOM mechanism addresses this problem by creating isolated subtrees attached to specific host elements. These shadow trees operate within a defined boundary, preventing unintended modifications to the global document structure.

A similar principle can be applied to structural program analysis. Shadow trees act as isolated structural representations where modifications and transformations can occur without altering the original program structure. This approach allows systems to perform localized structural comparisons or updates while preserving the integrity of the original

representation. Declarative shadow structures further improve efficiency by applying updates only to specific nodes rather than reconstructing entire tree representations.

2.5.4 Resource-Aware Decision Models

Efficient structural analysis of large program graphs requires mechanisms that minimize unnecessary computation and memory consumption. Resource-aware decision models provide a strategy for achieving this efficiency by focusing analysis on localized portions of the graph rather than performing exhaustive traversal of the entire structure.

Studies on graph execution efficiency demonstrate that localized node processing can significantly reduce memory usage while preserving structural relationships between program components (Zhi et al., 2023). Instead of analyzing every node globally, resource-aware algorithms prioritize nodes based on relevance or structural importance.

In the context of structural conformance analysis, this principle supports the use of localized AST node evaluation and targeted traversal strategies. By limiting analysis to structurally relevant regions of the program graph, systems can maintain scalability while still detecting deviations from expected architectural patterns.

2.5.5 Dependency Grammars as a Proxy for Code Semantics

Structural analysis techniques can also benefit from concepts derived from dependency grammars, which are widely used in natural language processing. Unlike phrase-structure grammars that represent sentences through nested hierarchical constituents, dependency grammars represent sentences as directed relationships between words. These relationships

typically link a governing word (head) to dependent words, forming a dependency graph rooted at a central element (Jurafsky & Martin, 2026).

This representation makes grammatical relationships explicit by encoding typed relationships such as subject, object, or modifier connections. Because these dependencies directly connect semantic roles to governing predicates, dependency grammars are often considered strong approximations of semantic structure, allowing systems to capture important relationships between linguistic elements more directly than traditional phrase-structure representations (Jurafsky & Martin, 2026).

A similar concept can be applied to program analysis. By representing program components as dependency relationships between operations, variables, and control structures, analysis systems can infer meaningful relationships between code elements without requiring full semantic interpretation. In this way, dependency-style representations provide a structural proxy that enables analysis tools to capture relationships between program components in a manner analogous to how dependency grammars represent relationships between words in natural language.

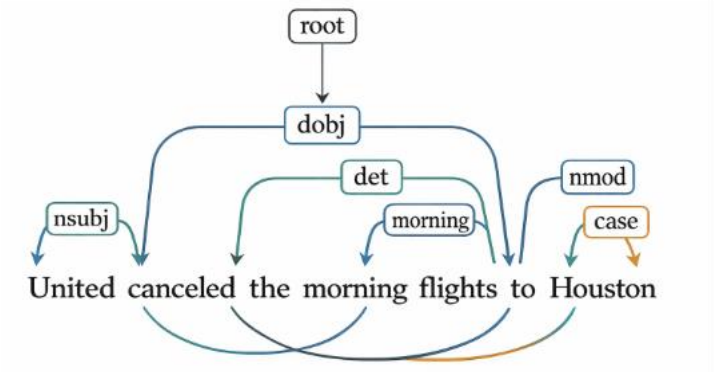


Figure 2. Dependency Parsing (Jurafsky & Martin, 2026)

2.5.6 Semantic Inference through Syntactic Heuristics

Although syntactic structures primarily represent program form rather than meaning, structural relationships can often provide useful approximations of semantic similarity. Research on structural comparison techniques has demonstrated that semantic correspondence between code fragments can be approximated through heuristic analysis of syntactic features.

Fan et al. (2021) introduced a hierarchical similarity evaluation framework that combines multiple structural and lexical similarity measures. These measures include metrics such as identical token counts, parent node mapping relationships, and the longest common subsequence between code fragments. By combining these features, the system can estimate semantic similarity between structurally related code segments (A Structural Approach to Program Similarity Analysis).

Such heuristic approaches demonstrate that aspects of semantic equivalence can be approximated through structural relationships alone, without requiring dynamic execution or complex semantic modeling. This principle supports the use of structural heuristics as part of large-scale conformance analysis systems.

2.5.7 Cryptographic Hash Trees for Structural Integrity

Efficient detection of structural modifications within large program graphs can be achieved using cryptographic hash tree structures, commonly referred to as Merkle Trees. In a Merkle Tree, leaf nodes represent the cryptographic hashes of individual data elements, while parent nodes are generated by hashing the concatenated hashes of their child nodes.

This hierarchical hashing structure enables efficient verification of structural integrity. When a modification occurs in any leaf node, the resulting hash change propagates upward through the tree, ultimately altering the root hash. As a result, structural equivalence between two program representations can be verified by comparing their root hashes.

When applied to structural code analysis, Merkle Trees can form a Merkle Directed Acyclic Graph (Merkle DAG) in which program constructs are content-addressed through cryptographic hashes. This structure allows analysis systems to detect structural differences between program graphs without traversing the entire codebase. Instead, comparisons can be performed by examining hash values associated with relevant nodes.

2.5.8 Copy-on-Pin (COP) and Concurrency Issues

Large-scale structural analysis often requires concurrent processing of program representations. However, naive duplication of large Abstract Syntax Trees can lead to significant memory overhead and performance degradation in multi-threaded environments.

Operating systems address similar challenges using Copy-on-Write (COW) mechanisms, which allow multiple processes to share memory pages until modifications occur. This technique improves memory efficiency by delaying duplication until a write operation is required. However, issues arise when memory pages become pinned for device access or concurrent processing. Pinned pages cannot safely rely on traditional COW sharing because concurrent modifications may lead to inconsistencies or corruption in shared memory states. As a result, memory management mechanisms must ensure that pinned pages are handled carefully to maintain system correctness and stability (Hildenbrand et al., 2023).

To address this issue, Hildenbrand et al. (2023) proposed the Copy-on-Pin (COP) mechanism, which ensures that pinned memory pages are immediately duplicated rather than shared. By enforcing mutual exclusivity between pinned pages and shared memory regions, COP prevents concurrency conflicts that could otherwise arise under traditional COW behavior. This design allows systems to maintain memory safety while preserving the performance benefits of shared memory in multi-threaded environments (Hildenbrand et al., 2023).

When applied to structural program analysis, Copy-on-Pin enables systems to process large AST structures concurrently without introducing memory corruption or excessive duplication. This approach improves both performance and reliability when analyzing large collaborative repositories.

2.6 Evaluation and Checking

2.6.1 Confusion Matrix used in AST Correctness Checking

In evaluating the structural and semantic code comparison capabilities of various Graph Neural Network (GNN) architectures, performance is quantified using standard confusion matrix metrics—specifically Precision, Recall, and the F1-score. The Graph Matching Network (GMN) inherently excels at capturing syntactic patterns directly from standard Abstract Syntax Trees (ASTs), achieving a baseline precision of 0.986, a recall of 0.922, and an F1-score of 0.953. When enriched with its best-performing hybrid representation (AST + FA + CFG), the GMN shows only marginal improvements, maintaining a precision of 0.986 while slightly increasing recall to 0.925 and the F1-score to

0.954. Conversely, the Graph Convolutional Network (GCN) benefits significantly from the addition of Control Flow Graphs (CFG) and Data Flow Graphs (DFG), jumping from a baseline F1-score of 0.868 (precision: 0.910, recall: 0.829) to an improved F1-score of 0.900 (precision: 0.950, recall: 0.855). Similarly, the attention-based Graph Attention Network (GAT) improves from a baseline F1-score of 0.872 (precision: 0.904, recall: 0.842) to an F1-score of 0.901 (precision: 0.922, recall: 0.881) when utilizing the AST + CFG + DFG hybrid. Finally, the Graph Gated Neural Network (GGNN) demonstrates minimal performance variance; its baseline AST model yields a precision of 0.907, a recall of 0.890, and an F1-score of 0.900 , which only slightly shifts to a precision of 0.923, a recall of 0.884, and an F1-score of 0.903 when using the AST + CFG + DFG structure.

Another important challenge is the need to quantify evaluation results objectively. Without standardized evaluation metrics, it becomes difficult to determine whether a structural analysis method performs reliably. In empirical software engineering research, evaluation is commonly based on statistical measures derived from a confusion matrix, where outcomes are categorized as true positives, true negatives, false positives, and false negatives. These classifications enable the computation of performance indicators such as accuracy, precision, recall, and F1-score, which are widely used to evaluate automated analysis systems (Yeboah & Popoola, 2024). Such metrics provide a systematic framework for assessing whether structural conformance checking mechanisms correctly identify compliant and non-compliant code contributions.

2.6.2 KPI Memory Tracking and speed parsing parsing

To accurately validate theoretical algorithmic complexity, developers must bridge abstract concepts with practical, empirical measurements. Time complexity fundamentally measures how the execution time of an algorithm grows relative to the size of its input. To practically validate an O(n) assumption, researchers utilize std::chrono::high_resolution_clock, which is the most accurate clock provided by the C++ standard library for profiling execution time. By recording a specific timepoint immediately before and after a function executes, the exact elapsed duration can be calculated. Developers frequently use chrono to measure this delta time in order to evaluate and compare the performance of algorithms in the context of their theoretical time complexity. Therefore, by profiling the algorithm across varying input sizes and demonstrating that the delta time measured by std::chrono scales proportionally with the input, an O(n) linear time complexity can be empirically validated. In addition to execution time, evaluating the spatial efficiency of a conformance checking algorithm is critical for a comprehensive structural analysis. For an Abstract Syntax Tree (AST) representing source code, the memory footprint is directly proportional to the number of nodes (n) generated during the parsing phase. In C++, the exact memory allocated for a single, uniform node structure can be determined at compile-time using the sizeof operator. Because each node—typically containing token metadata, structural properties, and pointers to child nodes—occupies a strict, constant amount of memory, the total space required to construct the tree in the heap is essentially $n \times \text{sizeof}(\text{ASTNode})$. Since the sizeof operator yields a constant byte value (c), the overall memory allocation equation simplifies to $c \times n$. Therefore, the memory footprint scales linearly with the size of the input code, theoretically guaranteeing an O(n) space complexity.

Empirically, tracking the total node count and multiplying it by the sizeof the data structure validates that the algorithm's spatial requirements do not grow quadratically or exponentially as code complexity increases.

2.6.3 Use of Linear Regression

According to Mario F. Triola in Elementary Statistics (2021), linear regression is utilized to algebraically describe the relationship between two variables derived from a collection of paired sample data. In statistical modeling, this involves defining an explanatory variable (x)—in this case, the algorithm's input size represented by the node count—and a response variable (y), representing the execution time or memory footprint. These variables are used to formulate a regression equation, typically expressed as $y = b_0 + b_1x$. The graph of this equation produces a line of best fit, which is governed by the least-squares property to minimize the variance between the observed data points and the predicted model. By applying this probabilistic model to the benchmarking data, researchers can rigorously demonstrate whether the computational resources scale strictly in proportion to the input size, thereby statistically formalizing the validation of an O(n) time and space complexity.

2.6.4 Ensuring Benchmarking Reliability via Cochran's Formula

To guarantee that the algorithm is evaluated against a statistically robust number of test cases, establishing a conservative optimal sample size is critical. When the true population proportion or the exact variance of the target metrics is unknown prior to empirical testing, standard statistical practice dictates setting the estimated proportion (p) to

0.5 (Triola, 2021). This baseline represents the point of maximum variability ($p \times q = 0.5 \times 0.5 = 0.25$), which mathematically maximizes the required sample size in standard proportion estimation formulas. By assuming this maximum variance, the calculation yields a highly conservative, worst-case scenario limit for the required number of trials (Triola, 2021). Consequently, benchmarking this specific number of parsed code structures ensures that the paired sample data collected for the linear regression analysis maintains the desired confidence level and margin of error, making the subsequent computational complexity findings highly reliable and resistant to sampling bias.

2.7 Synthesis

The reviewed literature highlights the growing complexity of modern software development and the increasing need for reliable mechanisms that ensure structural consistency within collaborative repositories. Existing studies emphasize that maintaining architectural integrity is a critical challenge as software systems evolve through contributions from multiple developers and automated tools. Research on technical debt and software maintenance further indicates that inconsistencies in code structure, design patterns, and architectural decisions can accumulate over time, eventually reducing system maintainability and increasing the cost of future development activities (Moreschini et al., 2026; Saraiva et al., 2021). Effective tools and frameworks are therefore required to detect structural inconsistencies early and support developers in maintaining long-term code quality.

Recent advances in software engineering have introduced artificial intelligence–driven development tools and large language models capable of generating, analyzing, and transforming code. These technologies demonstrate strong capabilities in tasks such as code generation, program

understanding, and automated documentation. However, despite their usefulness in improving development productivity, AI-based systems often rely on probabilistic reasoning and statistical prediction, which can produce inconsistent outputs across executions. Because architectural conformance checking requires deterministic evaluation of structural relationships within source code, relying solely on probabilistic tools may not guarantee reliable validation of software architecture and design constraints (Chen et al., 2021; Hou et al., 2024).

Traditional static analysis tools provide a deterministic alternative by examining source code without executing it and identifying potential issues related to coding standards, bugs, and vulnerabilities. These tools are widely used in industry to support software quality assurance and program verification. Nevertheless, studies have shown that developers often face limitations when using static analysis systems, including usability challenges, difficulty interpreting warnings, and scalability concerns when analyzing large codebases (Nguyen Quang Do et al., 2020). While static analysis can effectively detect certain types of issues, it may not fully capture complex structural relationships between program components, particularly in large collaborative repositories.

To address these limitations, recent research has explored structure-aware program representations such as Abstract Syntax Trees (ASTs) and graph-based models. These representations allow program elements and their relationships—such as control flow, data flow, and syntactic hierarchy—to be modeled explicitly. Graph-based representations, in particular, enable deeper analysis of program structures and have been applied in tasks such as code similarity detection, clone detection, and cross-language code analysis (Fan et al., 2021; Wang et al., 2021; TehraniJamsaz et al., 2023; Zhang & Saber, 2025). However, many advanced graph-based approaches introduce significant computational overhead due to the complexity of the representations and the algorithms required to process them.

The literature therefore reveals a clear gap between existing approaches. On one hand, AI-based tools provide flexible and intelligent support for software development but lack deterministic guarantees required for strict architectural validation. On the other hand, static analysis and structural models provide deterministic evaluation but may suffer from scalability limitations when applied to large repositories. Furthermore, hybrid graph representations and machine learning-based methods improve structural modeling but often increase computational cost and memory requirements (Zhi et al., 2023; Zhang & Saber, 2025).

Taken together, these findings highlight the need for an approach that combines deterministic analysis with efficient structural modeling of source code. Such a solution should be capable of evaluating architectural conformance while maintaining computational efficiency suitable for large collaborative environments. The proposed research addresses this gap by introducing a graph-based algorithmic framework designed to support structural conformance checking through deterministic processing of program representations. By integrating formal parsing mechanisms with efficient graph-based operations, the proposed method aims to provide a scalable and reliable mechanism for analyzing code structure and identifying deviations from intended architectural patterns.

Chapter 3

METHODOLOGY

This chapter presents the methodology used in the design, development, and evaluation of CodiNeo, a web-based design-pattern learning support system for C++ source code. The system combines learning modules, C++ source-code analysis, design-pattern evidence detection, documentation-oriented outputs, testing support, and research evaluation tools. It is designed to support users in understanding software design-pattern concepts, recognizing supported design-pattern evidence in actual C++ implementation, and generating documentation-oriented explanations that assist code understanding.

This chapter discusses the type of research, research design, system overview, system architecture, development process, algorithmic analysis process, hardware and software requirements, evaluation methods, data gathering procedures, respondents of the study, sampling technique, and statistical treatment of data.

3.1 Type of Research

This study uses a developmental research design with descriptive-evaluative components. It is developmental because the main objective of the study is to design and develop a software-based system that supports design-pattern learning and C++ source-code understanding. The descriptive-evaluative component applies because the study also aims to assess the developed system based on its functionality, usability, output clarity, design-pattern evidence usefulness, documentation support, learning support, and technical maintainability. Since the system functions both as a learning-support tool and a source-code analysis platform, the evaluation considers not only

whether the system works technically, but also whether its outputs are understandable and useful to the intended users.

3.2 Research Design

This study follows a developmental system design composed of five major phases: needs identification and planning, system design, system development, system testing and integration, and system evaluation. These phases guide the development and assessment of CodiNeo as a design-pattern learning support system for C++ source code.

The first phase is needs identification and planning. In this phase, the researchers identify the problem addressed by the study, particularly the difficulty of connecting software design-pattern concepts with actual C++ source-code implementation. The researchers also define the intended users, system objectives, supported design-pattern scope, and evaluation requirements based on the needs of learners, interns, novice developers, and DEVCON Luzon as the study context.

The second phase is system design. In this phase, the researchers design the major components of the system, including the learner pathway, developer or studio pathway, admin and evaluation dashboard, database flow, and source-code analysis workflow. The researchers also identify how C++ source code will be submitted, processed, analyzed, displayed, documented, saved, and evaluated within the system.

The third phase is system development. In this phase, the researchers implement the system components based on the defined design. This includes the development of the learning module

interface, source-code submission interface, analysis result views, documentation output views, backend processing services, C++ analysis microservice, database persistence, testing support, survey and review collection, and admin monitoring features.

The fourth phase is system testing and integration. In this phase, the researchers verify whether the different system modules work together as intended. This includes checking user login, learner module access, source-code submission, backend validation, C++ analysis processing, design-pattern evidence display, documentation output generation, supported testing features, saved runs, survey or review submission, and admin dashboard monitoring.

The fifth phase is system evaluation. In this phase, the researchers gather and analyze data from selected users and system records to determine the usefulness of CodiNeo in supporting design-pattern learning and code understanding. The evaluation considers user feedback, system usability, output clarity, functional suitability, design-pattern evidence usefulness, documentation support, and relevant technical metrics recorded through the system.

This research design ensures that the study is not limited to developing the system only, but also includes testing and evaluating whether CodiNeo supports its intended learning and code-understanding purpose in the DEVCON Luzon context.

3.3 System Overview

CodiNeo is a web-based design-pattern learning support system developed to help users understand selected software design-pattern concepts and recognize how these patterns appear in

actual C++ source code. The system is designed for DEVCON Luzon as a learning and code-understanding support tool for learners, interns, novice developers, and developers who need assistance in studying design patterns, analyzing C++ code, and generating documentation-oriented explanations.

The system is composed of three major user-facing components: the Learner Side, the Developer or Studio Side, and the Admin or Research Dashboard Side. Each component supports a different part of the system's learning, analysis, and evaluation process.

The Learner Side provides learning modules about selected software design patterns. This part of the system introduces users to the purpose, structure, and basic implementation ideas of supported design patterns. It is intended for users who need conceptual guidance before applying design-pattern concepts to actual source-code analysis. Through this component, users can study design-pattern topics and prepare themselves to recognize pattern evidence in C++ code.

The Developer or Studio Side serves as the main source-code analysis environment of the system. In this component, users can enter, paste, or upload C++ source code for analysis. The system examines the submitted code, identifies supported design-pattern evidence, highlights relevant structural elements, presents pattern-related results, and generates documentation-oriented outputs. This side is intended for users who want to directly analyze C++ code and understand how supported design patterns are reflected in actual implementation.

The Studio Side includes several analysis-related features. It allows users to view detected design patterns, inspect code annotations, review class-level structures, examine pattern evidence, access documentation outputs, and use supported testing features where applicable. The system may also display diagnostics, unit-test targets, and structured reports based on the submitted source code. These outputs help users understand not only what pattern was detected, but also why the pattern was identified based on the code structure.

The Admin or Research Dashboard Side supports the monitoring and evaluation of the system during the research process. This component allows the researchers or administrators to view system activity, user accounts, analysis runs, reviews, survey responses, logs, audit records, pattern frequency data, unit-test accuracy, complexity data, and F1-related metrics. It also provides controls for managing tester accounts, review requirements, and research-related exports. This side is important because the system functions not only as a learning and analysis tool, but also as an instrument for collecting data needed in the evaluation of the study.

Overall, CodiNeo is designed to support the transition from design-pattern theory to practical C++ source-code understanding. By combining learning modules, evidence-based code analysis, documentation-oriented explanations, testing support, and research monitoring tools, the system provides a guided environment where users can study design patterns, apply those concepts to real code, and generate outputs that support learning and code comprehension.

3.4 Algorithm

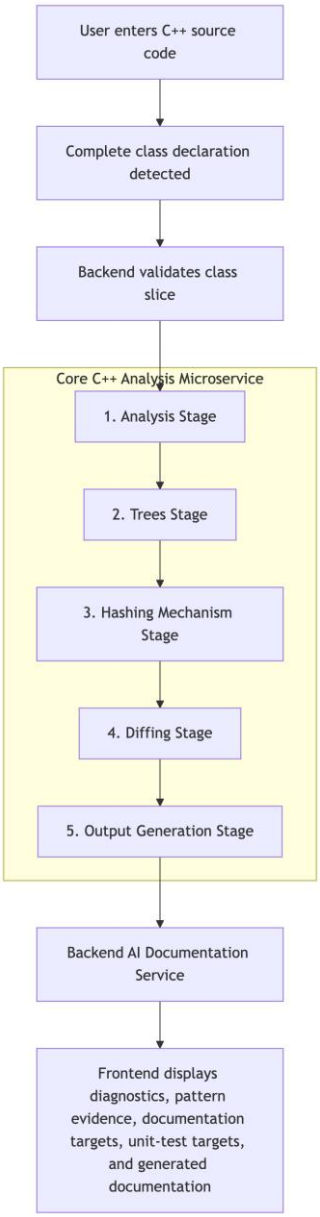


Figure 3. Internal Algorithm Processing Pipeline

The algorithmic analysis process of CodiNeo refers to the deterministic source-code analysis performed by the C++ microservice. This process is responsible for examining submitted C++ source code, extracting structural information, identifying supported design-pattern evidence, and

preparing structured outputs that can be used for documentation-oriented explanations and learning support.

The system's analysis process begins when a user submits C++ source code through the Developer or Studio Side of the application. The submitted input may come from a pasted source-code snippet, an uploaded source file, or a supported multi-file submission. Before analysis, the backend validates the submitted input by checking file names, file size, source-code content, and supported input limits. After validation, the backend forwards the source code to the C++ analysis microservice, which serves as the core analysis engine of the system.

The C++ microservice performs the main static and structure-based analysis. It reads the submitted source code and conducts a lexical scan to identify meaningful code elements such as keywords, identifiers, symbols, class declarations, method declarations, attributes, and relevant structural tokens. This stage prepares the submitted code for structural processing by separating source text into analyzable units.

After lexical scanning, the system constructs a class-level structural representation of the submitted code. The analysis focuses on complete class or struct declarations because the system is intended to identify design-pattern evidence from object-oriented source-code structure. During this stage, the system examines class names, attributes, methods, constructors, access modifiers, object creation logic, and relationships that may indicate supported design-pattern structures.

Once the structural representation is prepared, the system performs pattern dispatch. In this stage, the analyzed class-level structure is compared against the supported design-pattern definitions included in the prototype. Each supported design pattern has corresponding structural indicators that may include method behavior, object creation patterns, class responsibilities, naming evidence, relationship evidence, or implementation-use evidence. The system does not claim to detect all existing design patterns; rather, it identifies evidence for the design patterns supported by the implemented detection logic.

After pattern dispatch, the system performs evidence binding. This process connects detected pattern evidence to specific code units, such as classes, methods, attributes, or relevant source-code lines. Evidence binding is important because the system is designed not only to report that a pattern may be present, but also to show why the pattern was detected. By binding evidence to actual code structures, the system can present understandable results that help users connect design-pattern concepts with actual C++ implementation.

The analysis process also applies hash and link analysis where applicable. Hashing is used to maintain stable references for analyzed files, classes, functions, nodes, or evidence records. These references help preserve identity during analysis, result generation, documentation anchoring, and report preparation. Through this mechanism, the system can organize detected structures and connect analysis results with their corresponding source-code locations.

After design-pattern evidence is detected and organized, the system prepares documentation targets and possible unit-test targets. Documentation targets identify the parts of the analyzed code

that may require explanation, such as important classes, methods, responsibilities, or detected pattern evidence. Unit-test targets, when available, identify code areas that may be used for supported validation or testing features. These targets are not intended to fully verify arbitrary programs, but they provide additional support for selected design-pattern cases.

The final output of the microservice is a structured analysis report. This report may include detected design-pattern evidence, highlighted code units, diagnostics, documentation targets, possible unit-test targets, structured metadata, and JSON-formatted analysis results. The backend receives these results and prepares them for display in the frontend. If optional AI-assisted documentation is configured, the backend may also use the structured analysis results to request beginner-friendly explanations or documentation-style commentary. However, the core design-pattern detection remains based on the deterministic analysis performed by the C++ microservice.

Overall, the algorithmic analysis process follows this sequence:

1. **Source-code submission**

The user submits C++ source code through pasted input, file upload, or supported multi-file input.

2. **Input validation**

   The backend checks the submitted file name, size, content, and supported input limits.

3. **Lexical scanning**

   The C++ microservice identifies source-code tokens and structural elements.

4. **Class-level structural representation**

   The system constructs a structural view of complete class or struct declarations.

5. **Pattern dispatch**

The system checks the analyzed structure against supported design-pattern definitions.

6. **Evidence binding**

   Detected pattern evidence is connected to specific classes, methods, attributes, or source-code locations.

7. **Hash and link analysis**

   Stable references are created for analyzed files, code units, and evidence records.

8. **Documentation and test-target preparation**

   The system identifies code units that may be used for documentation-oriented explanations and supported validation checks.

9. **Structured report generation**

   The system returns detected pattern evidence, diagnostics, documentation targets, possible unit-test targets, and structured JSON results to the backend.

10. **Frontend presentation**

    The frontend displays the analysis results through pattern cards, highlighted code evidence, documentation outputs, diagnostics, and supported testing or review features.

Through this process, the algorithm supports the main purpose of CodiNeo: helping users understand how selected design-pattern concepts appear in actual C++ source code, it provides evidence-based outputs that support design-pattern learning, code understanding, and documentation-oriented explanation.

3.4.1 Time Complexity Analysis

The time complexity of the CodiNeo source-code analysis process depends on the total size of the submitted C++ source code, the number of relevant structural elements extracted from the code, and the number of supported design-pattern checks applied by the prototype. The analysis process includes input traversal, lexical scanning, class-level structural representation, pattern dispatch, hash and link generation, documentation-target preparation, and structured report generation.

For this study, the total analyzed input size is represented as N. If the system analyzes multiple submitted files or code units, the total input size may be represented as the sum of the sizes of all analyzed files:

$$\sum_{i=1}^{K} L_i \approx N$$

Equation 1. Time Complexity of Simultaneous Hashing and Traversal

where K represents the number of submitted files or analyzed code units, and L_i represents the number of relevant tokens or structural elements in each file. This means that the total amount of work performed by the analyzer is based on the combined size of all submitted source-code inputs.

The lexical scanning stage processes the submitted source code sequentially. Each relevant token, symbol, identifier, keyword, or structural marker is examined once during scanning. Since the analyzer traverses the input in a linear manner, the lexical scanning stage has a time complexity of:

$$O(N)$$

Equation 2. Time Complexity of Lexical Scanning

After lexical scanning, the system constructs a class-level structural representation of the submitted code. This stage organizes complete class or struct declarations, methods, attributes, constructors, relationships, and implementation-use evidence. Since each extracted structural element is processed and placed into the class-level representation, this stage also has a time complexity of:

$$O(N)$$

Equation 3. Time Complexity of construction of structural representation

The pattern dispatch stage compares the extracted structural evidence against the supported design-pattern definitions implemented in the prototype. If P represents the number of supported design-pattern checks, then the theoretical cost of checking the analyzed structure against the pattern catalog may be represented as:

$$O(P \times N)$$

Equation 4. Time Complexity of pattern dispatch stage

However, in the implemented prototype, the number of supported design patterns is fixed and limited. Because P does not grow dynamically with the submitted input size, it may be treated as a constant. Therefore, the practical time complexity of the pattern dispatch stage can be simplified as:

$$O(N)$$

Equation 5. Time Complexity of construction of structural representation if simplified

The hash and link analysis stage creates stable references for analyzed files, classes, functions, nodes, or evidence records. These references help connect detected design-pattern evidence to specific source-code units and documentation anchors. The hashing mechanism may be represented as:

$$H_{\{node\}} = std :: hash < std :: string > (file_{path} + "|" + owner_{scope} + "|" + function_{name} + "|" + parameter_{hint})$$

Equation 6. Hashing Mechanism of C++ std::hash

Each hash is generated from a bounded string representation of a code unit or structural element. Since hashing is applied across the relevant analyzed elements, and hash insertion or lookup is generally treated as constant time on average, the hashing and link-generation stage has an average time complexity of:

$$O(N)$$

Equation 7. Time Complexity of hashing and link-generation stage

The documentation-target and report-generation stages depend on the number of detected evidence records, diagnostics, documentation targets, possible unit-test targets, and structured output entries. Let E represent the number of detected evidence entries. Since these outputs are derived from the analyzed source-code structure, the report-generation stage may be expressed as:

$$O(N + E)$$

Equation 7. Time Complexity of report generation stage

Because E is bounded by the analyzed structural elements under the supported prototype scope, this stage can also be treated as linear relative to the analyzed input.

Therefore, the overall time complexity of the core analysis process can be expressed as:

$$T(N) = O(N) + O(P \times N) + O(N) + O(N + E)$$

Equation 8. Total Time Complexity of the algorithm

Since P is fixed in the prototype and E is bounded by the extracted structural elements, the practical time complexity simplifies to:

$$T(N) = O(N)$$

Equation 9. Total Time Complexity expected

This means that, under the supported input conditions of the prototype, the expected running time of the core source-code analysis process grows linearly with the size of the submitted C++ source-code structure. Larger inputs with more files, classes, methods, attributes, and relationships may require more processing time, but the analysis remains primarily dependent on the total number of relevant structural elements processed by the system.

The AI-assisted explanation layer and testing/GDB runner layer are not included in the core time complexity of the deterministic analyzer. These components operate after the structural analysis process and may vary depending on external configuration, provider

response time, or selected test cases. Therefore, the stated time complexity applies only to the deterministic C++ source-code analysis process of the system.

3.4.2 Space Complexity

The space complexity of the CodiNeo source-code analysis process depends on the amount of submitted C++ source code, the number of lexical tokens extracted, the number of structural elements generated, the number of hash or link references created, and the size of the output records prepared by the system. These output records may include detected design-pattern evidence, diagnostics, documentation targets, possible unit-test targets, and structured analysis reports.

The analyzed input size is represented as N, where N refers to the total number of relevant tokens or structural elements extracted from the submitted C++ source code. If the system analyzes multiple submitted files or code units, N represents the combined size of all analyzed inputs.

The system first requires memory for storing the submitted source-code input and the lexical tokens produced during scanning. Since the number of tokens is proportional to the size of the submitted source code, this stage requires:

$$O(N)$$

Equation 10. Space Complexity of submitted source code

After lexical scanning, the system constructs a class-level structural representation of the submitted code. This representation may include class or struct declarations, methods, attributes,

constructors, relationships, and implementation-use evidence. Since these structural elements are derived from the submitted source code, the memory required for this representation also grows linearly with the input size:

$$O(N)$$

Equation 11. Space Complexity of Time Complexity of construction of structural representation

The system also generates hash and link references to maintain stable identities for analyzed files, classes, functions, nodes, and evidence records. These references allow the system to connect detected design-pattern evidence to specific source-code units and documentation anchors. The hashing mechanism may be represented as:

$$H_{\{node\}} = std :: hash < std :: string > (file_{path} + "|" + owner_{scope} + "|" + function_{name} + "|" + parameter_{hint})$$

Equation 6. Hashing Mechanism of C++ std::hash

If K represents the number of generated hash or link references, then the memory requirement for this stage may be represented as:

$$O(K)$$

Equation 10. Space Complexity of generated hash or link references

In the implemented prototype, the number of hash or link references is dependent on the number of analyzed structural elements. Since K grows in relation to N, this component can be simplified as:

$$O(N)$$

Equation 11. Space Complexity of number of hash or link references

The pattern evidence records, documentation targets, diagnostics, possible unit-test targets, and structured report entries also require memory. Let E represent the number of detected evidence records and D represent the number of documentation-related output entries. The memory requirement for these outputs may be represented as:

$$O(E + D)$$

Equation 11. Space Complexity of the outputs

Since the evidence records and documentation outputs are derived from the analyzed source-code structure, they are bounded by the number of extracted elements under the supported prototype scope. Therefore, this component is also treated as linear relative to the analyzed input.

The total space complexity of the core analysis process can be expressed as:

$$S(Total) = S_{\{SourceInput\}} + S_{\{Tokens\}} + S_{\{StructuralRepresentation\}} + S_{\{HashReferences\}} + S_{\{EvidenceRecords\}} + S_{\{OutputReports\}}$$

Equation 12. Total Space Complexity of the Core Analysis

In Big-O notation, this may be represented as:

$$S(N) = O(N) + O(N) + O(N) + O(K) + O(E + D)$$

Equation 13. Total Space Complexity of the Core Analysis in Big-O notation

Since K, E, and D are derived from and bounded by the analyzed source-code structure in the supported prototype, the practical space complexity simplifies to:

$$S(N) = O(N)$$

Equation 12. Simplified Space Complexity

This means that the memory requirement of the core C++ analysis process grows linearly with the size of the submitted source-code structure. Larger inputs with more files, classes, methods, attributes, relationships, and detected evidence may require more memory, but the system's space usage remains mainly proportional to the number of relevant structural elements processed by the analyzer.

The database, saved runs, admin dashboard records, survey responses, logs, AI commentary, and GDB or testing results are not included in the core space complexity of the deterministic analyzer. These components belong to the broader application and research-evaluation layer. They may increase storage usage over time depending on the number of users, saved analysis runs, feedback entries, and logged events. Therefore, the stated space complexity applies only to the in-memory processing requirements of the core C++ source-code analysis process.

Project Design and System Architecture

The system is designed as a multi-part system composed of four main architectural layers: frontend, backend, microservice, and infrastructure. This separation of responsibilities allows the

system to organize user interaction, service orchestration, core analysis, and runtime execution into distinct layers.

The system architecture is based on the implementation blueprint documented under docs/Codebase. The documented folder structure represents the intended real system structure and serves as the basis for explaining the system's internal organization.

3.4.3 Activity Diagram

Figure 4. Activity Diagram

The activity diagram illustrates the overall workflow of CodiNeo from user access to learning, code analysis, documentation output, review submission, and research monitoring. The process begins when the user opens the system and logs in. After authentication, the system determines the user's pathway based on the selected role or access type.

If the user enters through the Learner Side, the system displays the available design-pattern learning modules. The user may select a design-pattern topic, study the module content, and proceed to the Studio Side when ready to apply the concept to actual C++ source code.

If the user enters through the Developer or Studio Side, the system allows the user to submit C++ source code through pasted input, file upload, or supported multi-file submission. The backend validates the submitted code and forwards it to the C++ microservice for analysis. The microservice performs lexical scanning, class-level structural analysis, supported design-pattern

evidence detection, hash and link generation, documentation-target identification, and structured report generation. The frontend then displays the detected patterns, highlighted evidence, diagnostics, documentation-oriented outputs, and supported testing options.

The user may review the results, resolve ambiguous pattern findings when applicable, run supported tests, and save or submit the analysis run. After submission, the system records the run details, review responses, survey answers, logs, and related evaluation data in the database.

If the user enters through the Admin or Research Dashboard Side, the system displays research monitoring features such as user accounts, analysis runs, reviews, surveys, logs, audit records, pattern frequency data, complexity data, unit-test accuracy, and F1-related metrics. The admin may review system activity, export survey data, or manage tester accounts.

Overall, the activity diagram shows how CodiNeo supports the movement from design-pattern learning to practical C++ source-code analysis and research evaluation.

3.4.4 Use Case Diagram

Figure 5. Use Case Diagram

The use case diagram presents the main interactions between the users and CodiNeo. The system has three primary actors: the Learner, the Developer, and the Admin. Each actor interacts with the system based on their role in the design-pattern learning, source-code analysis, and research evaluation process.

The Learner uses the system to access learning modules, study selected software design-pattern concepts, and proceed to the Studio Side when ready to apply the concepts to actual C++ source-code analysis. This actor represents users who need conceptual support before analyzing source code.

The Developer uses the system to submit C++ source code, run source-code analysis, view detected design-pattern evidence, inspect highlighted code structures, generate documentation-oriented outputs, run supported tests, save analysis runs, and submit review or feedback. This actor represents users who directly use the system for source-code understanding and documentation support.

The Admin uses the system to manage tester accounts, monitor user activity, view analysis runs, inspect reviews and survey responses, view logs and audit records, check system metrics, and export evaluation data. This actor represents the researchers or administrators who manage the system during the evaluation process.

The use case diagram therefore shows how CodiNeo supports three major functions: learning design-pattern concepts, analyzing C++ source code, and collecting evaluation data for the study.

3.4.5 Class Diagram

Figure 6. Class Diagram

The class diagram represents the main classes and relationships within CodiNeo. It shows how users interact with the learning module, source-code analysis, documentation, testing, review, survey, and admin monitoring features of the system.

The system has three primary user roles: Learner, Developer, and Admin. These roles are modeled as specialized types of the general User class. A learner can access design-pattern learning modules, a developer can submit C++ source code for analysis, and an admin can monitor system records, user activity, survey responses, logs, and evaluation metrics.

The LearningModule and DesignPatternLesson classes represent the learning side of the system. These classes store the selected design-pattern topics, descriptions, examples, and learning content that help users understand design-pattern concepts before applying them to actual C++ source-code analysis.

The SourceSubmission, AnalysisRun, and AnalysisResult classes represent the developer or studio side of the system. A developer can create a source submission by entering, pasting, or uploading C++ source code. Each submission may produce an analysis run, and each analysis run contains analysis results generated by the C++ analysis process.

The PatternEvidence class stores the detected design-pattern evidence found in the submitted source code. It is connected to the AnalysisResult because each result may include one or more detected design-pattern evidence records. The DocumentationOutput class stores the documentation-oriented explanations generated from the analysis result and detected

evidence. The TestResult class stores supported testing or validation results, such as compile checks or unit-test outputs for selected cases.

The ReviewResponse and SurveyResponse classes represent the evaluation data submitted by users. These records help the researchers assess system usefulness, clarity, usability, design-pattern learning support, and code understanding support.

The AdminDashboard, SystemLog, AuditLog, and EvaluationMetric classes represent the admin and research monitoring side of the system. These classes allow administrators and researchers to monitor analysis runs, user activity, logs, survey data, pattern frequency, complexity data, unit-test accuracy, and F1-related metrics.

Overall, the class diagram shows how CodiNeo connects design-pattern learning, C++ source-code analysis, documentation-oriented explanation, testing support, and research evaluation into one integrated system.

3.4.6 System Architecture

Figure 5. Layered Microservices Architecture of CodiNeo

CodiNeo follows a layered microservices architecture. The system is organized into separate layers that handle user interaction, backend orchestration, C++ source-code analysis, data persistence, optional AI-assisted documentation, testing support, and research evaluation. This

architecture allows each layer to perform a specific responsibility while still working together as one integrated design-pattern learning and documentation generation system.

The architecture is considered layered because the system separates its responsibilities into different functional layers. The frontend layer handles the user interface, the backend layer manages request processing and coordination, the C++ microservice performs the core deterministic analysis, the database layer stores system and research data, and the admin layer supports monitoring and evaluation. It is also considered microservices-based because the C++ analysis engine operates as a separate service from the main backend application.

The system architecture is composed of the following major layers: Frontend Layer, Backend or Application Server Layer, C++ Microservice Analysis Layer, Database and Persistence Layer, AI Documentation Layer, Testing and GDB Runner Layer, and Admin and Evaluation Layer.

3.4.6.1 Frontend Layer

The frontend layer serves as the user-facing interface of CodiNeo. It provides access to the learner side, developer or studio side, and admin dashboard. Through this layer, users can log in, access learning modules, submit C++ source code, view detected design-pattern evidence, inspect highlighted code structures, read documentation-oriented outputs, run supported tests, answer reviews or surveys, and access admin monitoring features when authorized.

For the learner side, the frontend displays design-pattern learning modules that introduce selected pattern concepts, examples, and explanations. For the developer or studio side, the frontend provides the source-code input area, analysis result tabs, pattern evidence cards, documentation tab, testing tab, and review prompts. For the admin side, the frontend displays users, analysis runs, reviews, surveys, logs, audit records, complexity data, unit-test accuracy, pattern frequency, and F1-related metrics.

3.4.6.2 Backend Layer or Application Server Layer

The backend layer serves as the main orchestration layer of the system. It receives requests from the frontend, validates submitted input, manages authentication, coordinates source-code analysis, handles saved runs, processes review and survey data, and communicates with supporting services such as the C++ microservice, database, optional AI service, and testing runner.

When a user submits C++ source code, the backend checks the submitted file name, file size, content, and supported input limits. After validation, it forwards the source code to the C++ microservice for analysis. Once the analysis result is returned, the backend organizes the output into a format that can be displayed by the frontend. It also manages whether the analysis run remains temporary, is saved by the user, or is submitted as part of the research evaluation workflow.

3.4.6.3 C++ Microservice Analysis Layer

The C++ microservice analysis layer is the core technical component of CodiNeo. It performs the deterministic source-code analysis used to identify supported design-pattern evidence. This layer is separate from the backend so that the code analysis logic remains isolated, maintainable, and focused on C++ structural processing.

The microservice reads the submitted C++ source code and performs lexical scanning, class-level structural analysis, pattern dispatch, evidence binding, hash and link generation, documentation-target identification, possible unit-test target extraction, and structured report generation. It returns the analysis results to the backend in a structured format, which may include detected pattern evidence, diagnostics, source-code references, documentation targets, possible unit-test targets, and JSON-formatted reports.

3.4.6.4 Database and Persistence Layer

The database and persistence layer stores the system and research-related records. This includes user accounts, roles, tester account claims, analysis runs, analysis results, review responses, survey responses, manual pattern decisions, logs, audit records, test-run events, and evaluation-related data.

This layer is important because CodiNeo functions not only as a learning and analysis tool, but also as a research instrument. By storing analysis runs, feedback, surveys, reviews, logs, and metrics, the system allows the researchers to evaluate how useful the system is in supporting design-pattern learning and C++ code understanding.

3.4.6.5 AI Documentation Layer

The AI documentation layer provides optional explanation support. When configured, this layer can generate beginner-friendly commentary, pattern explanations, documentation suggestions, or unit-test planning notes based on the structured analysis results produced by the C++ microservice.

The AI documentation layer is not responsible for detecting design patterns. The detection process remains dependent on the deterministic C++ analysis microservice. The AI layer only helps transform the structured analysis results into more readable documentation-style explanations when available.

3.4.6.6 Testing and GDB Runner Layer

The testing and GDB runner layer provides supported validation checks for selected code cases. This layer may compile and execute submitted or generated code under controlled conditions. It can be used to check whether selected C++ code examples compile or pass relevant test cases.

This layer is treated as an additional support feature and not as a full program verification system. It does not guarantee the complete correctness, security, memory safety, or runtime behavior of arbitrary submitted programs. Instead, it supports selected validation cases related to the prototype's design-pattern analysis and evaluation workflow.

3.4.6.7 Admin and Evaluation Layer

The admin and evaluation layer supports research monitoring, evaluation, and data management. Through the admin dashboard, authorized users can view user accounts, analysis runs, survey responses, review submissions, logs, audit records, pattern frequency, unit-test accuracy, complexity data, score distributions, and F1-related metrics. The admin dashboard may also provide controls for managing tester accounts, exporting evaluation data, and reviewing system activity.

This layer is important because it allows the researchers to monitor how users interact with the system, collect feedback, inspect system behavior, and gather data needed for evaluating the prototype.

3.4.6.8 Architecture Flow

The system architecture begins when a user accesses CodiNeo through the frontend. A learner may study design-pattern learning modules, while a developer may proceed directly to the studio and submit C++ source code for analysis. The frontend sends user requests to the backend, which validates the input and coordinates the required services. For source-code analysis, the backend forwards the submitted code to the C++ microservice. The microservice analyzes the code and returns structured results. The backend then prepares the results for display, stores relevant data in the database, and may coordinate optional AI explanation or testing features. The frontend displays the final outputs to the user, while the admin dashboard records and presents evaluation data for research monitoring.

Overall, the layered microservices architecture of CodiNeo supports separation of concerns, maintainability, and research evaluation. Each layer has a defined role: the frontend supports interaction, the backend coordinates processes, the C++ microservice performs deterministic analysis, the database preserves records, the AI layer supports explanation, the testing layer provides selected validation checks, and the admin layer supports monitoring and evaluation.

3.5 Sequence diagram

Figure 6. Sequence Diagram of the CodiNeo

The sequence diagram illustrates how CodiNeo processes a user's interaction from login, pathway selection, source-code analysis, output generation, review submission, and admin monitoring. Since the system has three main actors, the sequence diagram focuses on the interaction between the Learner, Developer, Admin, frontend, backend, C++ microservice, database, optional AI documentation service, and test runner.

For the Learner, the sequence begins when the user logs in and accesses the learning module page. The frontend requests the available design-pattern lessons from the backend. The backend retrieves the module data and returns it to the frontend, allowing the learner to study selected design-pattern concepts. After studying, the learner may proceed to the Studio Side to apply the concept to actual C++ source-code analysis.

For the Developer, the sequence begins when the user logs in and submits C++ source code through the Studio Side. The frontend sends the submitted code to the backend. The backend validates the input, checks file size and supported limits, and forwards the code to the C++ analysis microservice. The microservice performs lexical scanning, structural analysis, supported design-pattern evidence detection, hash and link generation, documentation-target preparation, and structured report generation. The backend receives the analysis result and returns it to the frontend, where the developer can view detected patterns, highlighted evidence, diagnostics, documentation-oriented outputs, and supported testing options.

If optional AI documentation is configured, the backend may send the structured analysis result to the AI documentation service. The AI service returns documentation-style explanations or beginner-friendly commentary, which are then displayed in the frontend. If the user runs supported tests, the backend sends the request to the test runner and stores the returned test result. When the user saves the run, submits a review, or answers a survey, the backend stores the data in the database.

For the Admin, the sequence begins when the admin logs in and opens the admin dashboard. The frontend requests dashboard data from the backend. The backend retrieves users, analysis runs, reviews, surveys, logs, audit records, pattern frequency data, complexity data, unit-test accuracy, and F1-related metrics from the database. These records are returned to the frontend and displayed in the admin dashboard for monitoring and evaluation.

Overall, the sequence diagram shows how CodiNeo connects the learner workflow, developer analysis workflow, and admin evaluation workflow into one integrated system.

3.6 Methods in Developing Software

Figure 3. Iterative and Risk-Driven Development Process

The development of CodiNeo followed an iterative and modular software development approach supported by CI/CD practices. This method was used because the system is composed of several connected modules, including the learner side, developer or studio side, backend application server, C++ analysis microservice, database and persistence layer, testing support, and admin dashboard. By developing the system in modules, the researchers were able to build, test, integrate, and refine each component while maintaining the overall purpose of supporting design-pattern learning and C++ source-code understanding.

The development process was organized into the following phases: requirements analysis, system design, module development, CI/CD and version control, system integration, system testing, and refinement.

3.6.1 Requirement Analysis

The first phase focused on identifying the needs of the intended users and the study context. The researchers determined that the system needed to support users in understanding selected software design-pattern concepts and recognizing how these patterns appear in actual C++ source

code. The system also needed to provide documentation-oriented outputs, supported testing features, and research evaluation tools.

During this phase, the researchers identified three main user roles: Learner, Developer, and Admin. The Learner role required access to design-pattern learning modules. The Developer role required access to the source-code analysis studio, where users could submit C++ code, view detected pattern evidence, and generate documentation-oriented explanations. The Admin role required access to monitoring and evaluation features such as users, analysis runs, reviews, surveys, logs, metrics, and exports.

3.6.2 System Design

The second phase involved designing the structure and workflow of CodiNeo. The researchers designed the system using a layered microservices architecture to separate responsibilities across the frontend, backend, C++ microservice, database, optional AI documentation layer, testing layer, and admin evaluation layer.

The system design also included the learner workflow, developer workflow, and admin workflow. The learner workflow was designed to support conceptual learning through design-pattern modules. The developer workflow was designed to support C++ source-code submission, analysis, evidence display, documentation output generation, and supported testing. The admin workflow was designed to support research monitoring, survey review, user management, log inspection, and evaluation-data export.

3.6.3 Module Development

The third phase focused on developing the system modules. The frontend module was developed to provide the user interface for the learner side, studio side, and admin dashboard. The learner side presents selected design-pattern learning modules. The studio side allows users to submit C++ source code, view detected design-pattern evidence, inspect highlighted code structures, access documentation-oriented outputs, and run supported tests where applicable. The admin side allows authorized users to monitor user activity, analysis runs, survey responses, reviews, logs, audit records, and evaluation metrics.

The backend module was developed as the system's application server and orchestration layer. It handles authentication, input validation, source-code submission, communication with the C++ microservice, saved analysis runs, survey and review records, testing requests, and admin dashboard data. The backend also manages how analysis results are returned to the frontend and how research-related records are stored for evaluation.

The C++ microservice module was developed as the core analysis engine of the system. It performs lexical scanning, class-level structural analysis, supported design-pattern evidence detection, hash and link generation, documentation-target identification, possible unit-test target extraction, and structured report generation. This module separates the analysis logic from the web interface and backend orchestration.

Additional modules were also developed to support the research workflow. These include the database and persistence module for storing user records, analysis runs, reviews, surveys, logs,

and metrics; the testing or GDB runner module for selected validation checks; the optional AI documentation module for explanation support when configured; and the admin dashboard module for monitoring and exporting evaluation data.

3.6.4 CI/CD and Version Control

The fourth phase involved the use of version control and CI/CD practices to manage development and deployment. The researchers used GitHub branches to separate feature development, bug fixes, and system updates. Changes were submitted through pull requests, allowing the team to review updates before merging them into the main development branch.

After approved changes were merged, the deployment workflow automatically built and deployed the updated system. This helped the researchers test new features continuously, verify whether recent changes affected existing functionality, and identify errors early during development. The CI/CD workflow also supported collaboration among the researchers by providing a controlled process for integrating and deploying system updates.

3.6.5 System Integration

The fifth phase involved integrating the developed modules into a complete working system. The frontend was connected to the backend through API requests. The backend was connected to the C++ microservice for source-code analysis and to the database for storing users, runs, reviews, surveys, logs, and evaluation records. The testing layer and optional AI documentation layer were also connected as supporting services where applicable.

Integration was important because the system depends on coordination among multiple components. For example, when a developer submits C++ source code, the frontend sends the request to the backend, the backend validates the input, the C++ microservice performs the analysis, the backend organizes the returned report, and the frontend displays the detected evidence and documentation outputs. If the user saves the run, answers a survey, or submits a review, the backend stores the related records in the database for evaluation.

3.6.6 System Testing

The sixth phase involved testing the functionality of the system modules and their integration. The researchers checked whether users could log in, access learning modules, submit C++ source code, receive analysis results, view detected design-pattern evidence, inspect documentation-oriented outputs, run supported tests, submit reviews or surveys, and save analysis runs. The admin dashboard was also tested to verify whether user records, runs, responses, logs, metrics, and exports could be accessed properly.

Testing was also used to check whether invalid or unsupported source-code submissions were handled properly through validation messages. This helped ensure that the system could respond appropriately when submitted code was incomplete, unsupported, or outside the prototype's accepted input conditions.

3.6.7 Refinement and Improvement

The final phase involved improving the system based on testing results, observed errors, and user feedback. Refinements were made to improve the clarity of the user interface, the

organization of analysis results, the presentation of design-pattern evidence, the documentation output flow, the testing experience, and the admin dashboard's evaluation features.

Through this iterative process, the researchers were able to improve the system gradually while ensuring that each component remained aligned with the study objectives. The combination of modular development and CI/CD practices allowed CodiNeo to be developed, tested, deployed, and refined continuously throughout the research process.

3.7 Hardware and Software Requirements

The hardware and software requirements define the minimum tools and environment needed to develop, run, and evaluate the system.

3.7.1 Software Requirements

CodiNeo requires several software tools and technologies to support its frontend interface, backend services, C++ analysis engine, database layer, testing features, and deployment workflow.

The following table summarizes the software requirements of the system:

Component	Software / Tool	Purpose
Frontend Development	React with Vite	Builds the learner side, studio side, and admin dashboard

Backend Development	Node.js, TypeScript, Express	Latest stable Node.js
C++ Analysis Microservice	C++17-compliant compiler	Handles APIs, authentication, validation, orchestration, and system services
C++ Analysis Engine	C++17, GCC/G++ or Clang	Compiles and runs the C++ microservice analysis engine
Database	SQLite / better-sqlite3	Stores users, runs, surveys, reviews, logs, metrics, and evaluation data
API Communication	RESTful API	Allows the frontend, backend, and services to communicate
Testing Support	GDB/Test Runner, Docker where applicable	Supports selected compile/test validation checks
Version Control	Git and GitHub	Manages branches, commits, pull requests, and collaboration
Deployment	CI/CD-supported automatic deployment	Builds and deploys system updates after approved changes
Browser	Modern web browser	Modern web browser

Table 2. Software Specification

3.7.2 Hardware Requirements

The hardware requirements refer to the minimum and recommended specifications needed to develop, run, and test the CodiNeo prototype. Since the system includes a web frontend, backend server, C++ microservice, database, and optional testing services, the development machine should have enough processing power and memory to run multiple components at the same time.

A computer with at least a modern multi-core processor, sufficient RAM, and available storage is required. The system can be developed and tested on a Windows, macOS, or Linux environment, provided that the required development tools, compilers, and runtime dependencies are installed.

The following table summarizes the hardware requirements:

Component	Minimum Requirements	Recommended Requirement
Processor	Dual-core processor	Quad-core processor or higher
RAM	8 GB	16 GB or higher
Storage	10 GB available space	20 GB or higher available space
Operating System	Windows, macOS, or Linux	Windows, macOS, or Linux with development tools installed

Network	Stable internet connection	Stable internet connection for GitHub, deployment, and API testing
Browser	Updated modern browser	Updated Chrome, Edge, Firefox, or Safari

Table 3. Hardware Specification List

3.8 Methods in Evaluating the System

The system will be evaluated based on its ability to perform class-level C++ source-code analysis and produce useful outputs for documentation generation, design-pattern understanding, unit-test target generation, and code understanding support. The evaluation will include internal system testing, expert software quality evaluation, and user-based questionnaire evaluation.

The evaluation focuses on the following areas:

1. functional suitability;
2. performance efficiency;
3. interaction capability or usability;
4. reliability;
5. maintainability;
6. documentation clarity;
7. design-pattern evidence usefulness;
8. onboarding and learning support.

These evaluation areas are appropriate because the proposed system is both a software-

based analysis tool and a learning-support system. Therefore, the evaluation must consider not only technical functionality but also whether the system outputs are understandable and useful to the intended users.

3.8.1 Software Evaluation Based on ISO Quality

This study uses selected ISO/IEC 25010 quality characteristics to evaluate the software quality of the system. The selected characteristics are chosen based on their relevance to the proposed system as a documentation generation and source-code analysis platform.

Sequence	ISO Quality Characteristic	Alignment to Algorithm Lifecycle Phase	Description of Evaluation Focus
1	Functional Suitability	Input, Analysis, and Output Phase	Assesses whether the system accepts C++ class or struct input, performs source-code analysis, detects supported design-pattern evidence, and produces documentation targets, unit-test targets, diagnostics, and reports.
2	Performance Efficiency	Analysis and Diffing Phase	Assesses whether the system processes class slices efficiently using scoped analysis, hashing, and partial regeneration planning.

3	Interaction Capability / Usability	Frontend and Result Presentation Phase	Assesses whether users can understand the editor, diagnostics, detected evidence, and generated documentation.
4	Reliability	Analysis and Report Generation Phase	Assesses whether the system produces stable and consistent results for similar source-code inputs.
5	Maintainability	Architecture and Extension Phase	Assesses whether the modular frontend, backend, C++ microservice, AI service, and infrastructure design support future updates and additional pattern definitions.

Table 5. ISO Standards Specifications

The ISO-based evaluation will be answered by the professional cohort, including experienced developers, mentors, instructors, or software engineering practitioners.

3.8.2 System Testing and Validation

System testing and validation will be conducted to determine whether the prototype performs according to its intended workflow. The testing process includes alpha testing, expert evaluation, and beta testing.

3.8.2.1 Alpha Testing

Alpha testing will be conducted by the researchers to verify the internal functionality of the system. This stage focuses on checking whether the frontend,

backend, C++ microservice, AI documentation service, and output generation components work together correctly.

Alpha testing will include the following:

1. checking whether the frontend accepts C++ source-code input;
2. checking whether the frontend detects complete class or struct declarations;
3. checking whether the backend validates incoming requests;
4. checking whether the C++ microservice performs lexical and structural analysis;
5. checking whether the system builds class-level subtree representations;
6. checking whether supported design-pattern evidence is detected;
7. checking whether documentation targets are generated;
8. checking whether unit-test targets are generated;
9. checking whether diagnostics and structured reports are produced;
10. checking whether the backend prepares AI documentation input;
11. checking whether AI-assisted documentation is returned; and
12. checking whether the frontend displays the results correctly.

The purpose of alpha testing is to confirm that the system workflow functions as intended before it is evaluated by external users and professional reviewers.

3.8.2.2 Expert Evaluation

Expert evaluation will be conducted with the professional cohort. The professional cohort may include mentors, instructors, experienced developers, or software engineering practitioners. Their role is to assess the technical quality of the system based on selected ISO/IEC 25010 quality characteristics.

The expert evaluation will focus on whether the system is functionally suitable, reliable, maintainable, efficient, and usable as a source-code analysis and documentation support system. The professional cohort will also assess whether the detected pattern evidence, documentation targets, unit-test targets, diagnostics, structured reports, and generated explanations are technically appropriate and useful.

3.8.2.3 Beta Testing

Beta testing will involve selected application users such as interns, novice developers, or software engineering students. These participants represent the intended users of the system because they are still developing their understanding of source-code structure, design patterns, documentation practices, and software implementation.

The purpose of beta testing is to evaluate whether the system is understandable and useful from the perspective of its intended users. The beta testing questionnaire will focus on the clarity of the interface, the usefulness of the generated documentation, the understandability of the detected design-pattern evidence, the relevance of unit-test targets, and the system's support for code understanding, internship onboarding, and design-pattern learning.

3.9 Data Gathering Procedures

The data gathering procedures are designed to collect information from system outputs, technical review, and user feedback. The study will gather both quantitative and qualitative data.

Quantitative data will be collected from ISO-based expert evaluation forms, user questionnaire responses, and system performance observations. Qualitative data will be collected from open-ended comments, expert feedback, and researcher observations during testing.

3.9.1 Expert Review for Software Quality Evaluation

Expert reviewers will evaluate the system using selected ISO/IEC 25010 quality characteristics. Their evaluation will focus on functional suitability, performance efficiency, interaction capability or usability, reliability, and maintainability.

The expert reviewers may include mentors, instructors, experienced developers, or software engineering practitioners. Their feedback will help determine whether the system outputs are technically appropriate and whether the system architecture supports future improvement and extension.

3.9.2 User Questionnaire for Application Users

Application users will answer a questionnaire after testing the system. These users may include interns, novice developers, or software engineering students. The questionnaire will measure the users' perception of the system in terms of clarity, usefulness, ease of use, documentation support, design-pattern learning support, and onboarding support.

The questionnaire may include items related to the following areas:

1. clarity of the user interface;
2. understandability of diagnostics;
3. clarity of generated documentation;

4. usefulness of detected design-pattern evidence;
5. usefulness of unit-test targets;
6. support for understanding unfamiliar C++ code;
7. support for connecting design-pattern concepts to implementation;
8. usefulness for internship onboarding or learning.

The results of the questionnaire will be summarized using weighted mean and verbal interpretation.

3.9.3 Observation using Key Performance Indicators (KPIs)

The researchers will observe the system during testing by recording selected system performance indicators. These indicators will help determine whether the system performs its analysis workflow efficiently.

The observed KPIs may include:

1. parsing or processing time;
2. memory usage;
3. number of processed class or struct declarations;
4. number of detected pattern-evidence entries;
5. number of generated documentation targets;
6. number of generated unit-test targets;
7. successful and failed analysis attempts.

These observations will be collected from backend logs, structured reports, and system-generated outputs. The purpose of KPI observation is to support the evaluation of performance efficiency and reliability.

3.10 Respondents of the Study and Sampling Techniques

The respondents of this study will be composed of two groups: the professional cohort and the application-user cohort. These groups are selected based on their relevance to the purpose of the system, which is to support C++ source-code analysis, documentation generation, design-pattern learning, and internship onboarding.

The first group is the professional cohort. This group consists of technical evaluators such as mentors, instructors, experienced developers, or software engineering practitioners. These respondents will assess the system based on selected ISO/IEC 25010 quality characteristics. Their evaluation will focus on functional suitability, performance efficiency, interaction capability or usability, reliability, maintainability, and the technical usefulness of the system outputs.

The second group is the application-user cohort. This group consists of application users such as interns, novice programmers, or software engineering students. These respondents represent the intended users of the system because they are still developing their understanding of C++ source-code structure, design patterns, and documentation practices. Their evaluation will focus on usability, clarity of diagnostics, usefulness of generated documentation, relevance of unit-test targets, and support for code understanding and design-pattern learning.

Since the exact population size of potential respondents cannot be determined with certainty, the study treats the target population as undefined. Respondents will be selected based on availability, programming background, and relevance to the evaluation of the system.

3.10.1 Sampling Technique

This study will use purposive sampling. Purposive sampling is a non-probability sampling technique in which participants are selected based on specific characteristics that are relevant to the objectives of the study.

For the professional cohort, selected respondents must have experience in software development, programming instruction, mentoring, code review, or software engineering practice. Their role is to provide informed feedback on the technical quality and usefulness of the system.

For the application-user cohort, selected respondents must have basic programming experience, preferably with object-oriented programming concepts. They may include interns, novice developers, or software engineering students who can evaluate whether the system is understandable and useful for code understanding, documentation support, and design-pattern learning.

This sampling technique is appropriate because the study requires respondents who can meaningfully assess the system's usability, documentation support, design-pattern evidence, and unit-test target generation.

3.10.1.1 Calculation of Margin of Error Based on Given samples

This study used the Cochran formula, which is used to determine the sample size of an unknown or infinite population, because the precise population size is unknown

(Triola, 2022). By computing the margin of error for a fixed sample mathematically instead of depending on estimations of the industry population that have not been verified, this approach avoids estimation bias.

The formula is expressed as:

$$n = \frac{Z^2 * p * (1 - p)}{e^2}$$

$$n = \text{sample size}$$

$$Z = Z - \text{score or confidence level}$$

$$p = \text{estimated proportion of the population}$$

$$\varepsilon = \text{margin of error}$$

Equation 4. Cochran's Formula

3.10.1.2 Justification for Maximum Variability (p = 0.5)

The population proportion (p) is a crucial part of this formula. This study used the common statistical precaution of setting p = 0.5 because there is currently no literature to precisely quantify the percentage of Filipino developers who have particular operational viewpoints (Triola, 2022).

Maximum variability, as defined theoretically, is the largest degree of uncertainty represented by this 50/50 split (Triola, 2022). The approach compels the investigation to account for the maximum degree of unpredictability by utilizing 50%.

Using the fixed sample size of 100 and these parameters:

$$100 = \frac{1.96 \times 0.5 \times (1 - 0.5)}{\varepsilon^2}$$

$$\varepsilon^2 = \frac{1.96 \times 0.5 \times 0.5}{100}$$

$$\varepsilon = \sqrt{\frac{0.49}{100}}$$

$$\varepsilon = 0.07$$

Equation 5. Margin of Error Calculation for Fixed Sample Size (n = 100)

The calculations rigorously demonstrate that a sample size of 100 respondents produces a margin of error of roughly 7% at a 95% confidence level, or Z score of 1.96, assuming maximum variability.

3.11 Statistical Treatment of Data

The data gathered from the study will be analyzed using both quantitative and qualitative methods. Quantitative data will come from expert evaluation forms, user questionnaire responses, system observations, and selected performance indicators. Qualitative data will come from open-ended comments, expert feedback, and researcher observations during system testing.

3.11.1 Weighted Mean for Expert and User Evaluation

The responses from the professional cohort and application-user cohort will be analyzed using weighted mean. This will be used to determine the overall rating of the system based on the evaluation criteria.

For the professional cohort, the weighted mean will be used to summarize the ISO-based evaluation results, including functional suitability, performance efficiency, interaction capability or usability, reliability, and maintainability.

For the application-user cohort, the weighted mean will be used to summarize the questionnaire results related to clarity, usefulness, documentation support, design-pattern learning support, onboarding support, and ease of use.

The weighted mean will be computed using the formula:

$$\bar{x}_w = \frac{\sum_{i=1}^{n}(w_i x_i)}{\sum_{i=1}^{n}(w_i)}$$

where as

$\bar{x}_w$ is the weighted mean variable

w_i is the allocated weighted value

x_i is the observed values.

The computed mean will be interpreted using a verbal interpretation scale based on the questionnaire rating system.

3.11.2 Quantitative Performance Profiling (KPIs)

Simple Linear Regression is used to statistically confirm that the Graph Memory Copy Algorithm follows its theoretical linear time and space complexity (O(n)). The regression model evaluates the relationship by treating the execution time or memory used as the dependent variable (y) and the input codebase size (number of AST nodes or tokens) as the independent variable (x).

$$y = B_0 + B_1 x + e$$

where:

$$y = \text{Execution Time or Memory Execution}$$

$$x = \text{input size}$$

$$B_0 = y - \text{intercept (baseline computational overhead)}$$

$$B_1 = \text{slope of regression line}$$

$$\varepsilon = \text{margin of error}$$

Equation 6. Linear Regression Testing of Linear Time and Space Complexity

System performance will be analyzed using selected Key Performance Indicators. These KPIs will include processing time, memory usage, successful analysis attempts, failed analysis attempts, and output generation counts.

Processing time will be used to observe how long the system takes to analyze a submitted C++ class or struct declaration. Memory usage will be used to observe the resource consumption of the C++ microservice during analysis. Successful and failed analysis attempts will be used to determine whether the system performs consistently under selected test cases.

These KPI results will be presented using tables and descriptive analysis. The results will support the evaluation of performance efficiency and reliability.

3.11.3 Confusion Matrix for Pattern-Evidence Detection

A confusion matrix may be used to evaluate the correctness of the system's design-pattern evidence detection. This evaluation will not measure code transformation or code

correction because the system does not rewrite, refactor, or modify the submitted source code. Instead, the confusion matrix will classify whether the system correctly detects or does not detect supported design-pattern evidence from selected C++ code samples.

The classifications are defined as follows:

Error Classification	System Action Description	Consequence of Failure
Type I Error (False Positive)	The system detects design-pattern evidence even though the source code does not sufficiently support it.	Valid code is unnecessarily modified, which may accidentally break the original logic.
Type II Error (False Negative)	The system fails to detect supported design-pattern evidence that is present in the source code.	Non-conforming legacy code is completely ignored and left unchanged, missing a necessary update.
True Positive (TP)	The system correctly detects supported design-pattern evidence that is present in the source code.	The detected issue is valid, the appropriate transformation is executed, and the resulting output structurally conforms to the expected specification.
True Negative (TN)	The system correctly does not detect design-pattern evidence when the submitted code does not contain supported evidence.	No unnecessary modifications are performed, and the original code remains unchanged while maintaining structural validity. There would only be an error message that would notify to the user that it does not conform to the expected code conformance.

Table 6. Guide description for junior developer cohorts in answering the given confusion matrix

The following metrics may be computed:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 - Score = 2 * \left(\frac{Precision * Recall}{Precision + Recall}\right)$$

Equation 7. Confusion Matrix Evaluation Formula (Triola, 2022).

These metrics will help evaluate the correctness of the pattern-evidence detection component. The results will be interpreted together with expert review and user feedback to determine whether the detected evidence is useful for documentation generation and design-pattern learning.

3.11.4 Qualitative Analysis of Comments and Feedback

Open-ended responses from expert reviewers and application users will be analyzed qualitatively. Comments will be grouped according to recurring themes such as interface

clarity, documentation usefulness, diagnostic clarity, design-pattern learning support, onboarding support, and suggested improvements.

This qualitative analysis will help explain the numerical results and provide additional insight into how the system can be improved.

Chapter 4

RESULTS

4.1 Overview

This chapter presents the results of the system evaluation conducted for the proposed Graph-Based and AST-Centered Code Analysis and Documentation System. The findings are organized according to the study's objectives, covering:

- Usability and functional evaluation (intern cohort)
- Expert validation (professional cohort)
- Quantitative performance metrics (KPIs)
- System effectiveness in supporting code understanding and onboarding

All results are presented using statistical summaries such as mean scores, percentages, and performance metrics, which will be populated after the completion of testing.

4.2 Respondent Profile

4.2.1 Intern Cohort (Student Users)

The intern cohort consists of participants who represent novice developers or students undergoing training in software development environments.

Profile Variable	Frequency	Percentage
Course/Program		
Programming Experience		
Familiarity with C++		

4.2.2 Professional Cohort (Experts)

The professional cohort consists of experienced developers, mentors, or practitioners aligned with DEVCON Luzon.

Profile Variable	Frequency	Percentage
Years of Experience		
Role		
Experience with Design Patterns		

4.3 Evaluation on ISO/IEC 25010 Quality Model

The system was evaluated using selected characteristics from ISO/IEC 25010, including:

- Functional Suitability
- Usability
- Performance Efficiency
- Reliability

- Maintainability

4.3.1 Functional Suitability

This section evaluates how well the system performs its intended functions, particularly:

- Code analysis
- Pattern detection
- Documentation generation
- Unit-test target generation

Indicator	**Mean**	**Interpretation**
Accuracy of Code Analysis		
Relevance of Documentation		
Relevance of Documentation		
Usefulness of Unit-Test Targets		

4.3.2 Usability

This measures how easy the system is to use for interns.

Indicator	**Mean**	**Interpretation**
Ease of Understanding Outputs		
Clarity of Documentation		
Learning Support Effectiveness		

4.3.3 Performance Efficiency

Performance is evaluated using system-level metrics such as:

- Response time
- Processing time
- Resource usage

Graph-based systems are known to involve computational trade-offs, especially in memory and traversal operations.

Metric	Mean	Interpretation
Average Processing Time		
Memory Usage		
Throughput		

4.3.4 Reliability

Metric	Mean	Interpretation
Consistency of Results		
Error Handling		

4.3.5 Maintainability

Metric	Mean	Interpretation
Ease of Extending System		
Code Modularity		

4.4 System Effectiveness in Code Understanding

This section evaluates how the system helps users interpret source code. Graph-based representations such as AST and code property graphs provide better structural understanding of programs by capturing syntax and semantics together.

Indicator	**Mean**	**Interpretation**
Understanding of Code Structure		
Identification of Key Components		
Comprehension of Program Logic		

4.5 Effectiveness of Documentation-Oriented Outputs

Indicator	**Mean**	**Interpretation**
Clarity of Generated Documentation		
Usefulness for Learning		
Alignment with Code Structure		

4.6 Effectiveness of Design Pattern Detection

Indicator	Mean	Interpretation
Accuracy of Pattern Identification		
Helpfulness in Learning Patterns		

4.7 Quantitative Performance Evaluation (KPIs)

This includes objective system metrics such as:

- Precision
- Recall
- Accuracy
- Confusion Matrix

Indicator	Mean
Accuracy	
Precision	
Recall	
F1 Score	

BIBLIOGRAPHY

Agarwal, A., Agarwal, A., Verma, D. K., Tiwari, D., & Pandey, R. (2023). A review on software development life cycle. International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 9(3), 384–388. https://doi.org/10.32628/CSEIT2390387

Alikhanifard, P., Papadakis, V., & Chatzigeorgiou, A. (2025). Accurate abstract syntax tree differencing: Language-aware design, benchmarking, and empirical assessment. *Empirical Software Engineering*. https://doi.org/10.1007/s10664-022-10176-0

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). *Software engineering for machine learning: A case study*. IEEE.

Chen, J., Bai, S., Wang, Z., Wu, S., Du, C., Yang, H., Gong, R., Liu, S., Wu, F., & Chen, G. (2025). Pre3: Enabling deterministic pushdown automata for faster structured LLM generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (pp. 11253–11267). https://doi.org/10.18653/v1/2025.acl-long.551

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., … Zaremba, W. (2021). *Evaluating large language models trained on code*.

Chung, J., Ko, R., Yoo, W., Onizuka, M., Kim, S., Kim, T.-W., & Shin, W.-Y. (2025). GraphCompliance: Aligning policy and context graphs for LLM-based regulatory compliance. *arXiv*. https://arxiv.org/abs/2510.26309

Copik, M., et al. (2023). Copy-on-Pin: The missing piece for correct copy-on-write.

Das, D., Al Maruf, A., Islam, R., Lambaria, N., Kim, S., Abdelfattah, A. S., Cerny, T., Frajtak, K., Bures, M., & Tisnovsky, P. (2020). Technical debt resulting from architectural degradation and code smells: A systematic mapping study.

Fan, Y., Xia, X., Lo, D., Hassan, A. E., Wang, Y., & Li, S. (2021). A differential testing approach for evaluating abstract syntax tree mapping algorithms. *arXiv*. https://arxiv.org/abs/2103.00141

Free Software Foundation. (2024). *The GNU C++ library (libstdc++) reference manual*. https://gcc.gnu.org/onlinedocs/libstdc++/

Gong, L., Elhoushi, M., & Cheung, A. (2024). AST-T5: Structure-aware pretraining for code generation and understanding. *arXiv*. https://arxiv.org/abs/2401.03003

Guda, D. P. (2025). Shifting security left in the insurance SDLC: A DevSecOps maturity model. International Journal of Environmental Sciences, 11(19s).

Hashimoto, T. (2025, September). Lecture 8: Self-attention and transformers [Lecture notes]. *CS224N/Ling284: Natural Language Processing with Deep Learning, Stanford University*.

Hildenbrand, D., Schulz, M., & Amit, N. (2023). Copy-on-pin: The missing piece for correct copy-on-write. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (pp. 176–191). https://doi.org/10.1145/3575693.3575716

Hou, Y., Chen, Z., Xu, B., & Lo, D. (2024). Large language models for software engineering: A systematic literature review.

ISO/IEC. (2023). *Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—Product quality model (ISO/IEC 25010:2023)*. International Organization for Standardization.

Krishnan, N. (2024). *AI agents: Evolution, architecture, and real-world applications*.

Li, R., Liang, P., Soliman, M., & Avgeriou, P. (2021). Understanding architecture erosion: The practitioners' perceptive. In *Proceedings of the 29th IEEE/ACM International Conference on Program Comprehension* (pp. 24–35). https://doi.org/10.1109/ICPC52881.2021.00037

Liu, J., Zeng, J., Wang, X., & Liang, Z. (2023). Learning graph-based code representations for source-level functional similarity detection. In *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering* (pp. 345–357). https://doi.org/10.1109/ICSE48619.2023.00040

Monaghan, B. D., & Bass, J. M. (2020). Redefining legacy: A technical debt perspective. In *Proceedings of the IEEE/ACM International Conference on Program Comprehension*. https://doi.org/10.1109/ICPC52881.2021.00037

Moreschini, S., Arvanitou, E.-M., Kanidou, E.-P., Nikolaidis, N., Su, R., Ampatzoglou, A., Chatzigeorgiou, A., & Lenarduzzi, V. (2026). The evolution of technical debt from DevOps to generative AI: A multivocal literature review.

Motwani, M., & Brun, Y. (2023). Understanding why and predicting when developers adhere to code-quality standards. *IEEE Transactions on Software Engineering*.

Muratdağı, T. (2024). Identifying technical debt and tools for technical debt management in software development.

Nesteruk, D. (2022). *Design patterns in modern C++20: Reusable approaches for object-oriented software design*. Apress. https://doi.org/10.1007/978-1-4842-7295-4

Nivre, J., et al. (2026). Dependency parsing.

Pareek, C. S. (2021). Driving agile excellence in insurance development through shift-left testing. International Journal for Multidisciplinary Research, 3(6).

Park, J., Lee, H., & Ryu, S. (2021). A survey of parametric static analysis. *ACM Computing Surveys, 54*(7), Article 149. https://doi.org/10.1145/3464457

Pranav, M., Madhesh, I., Lenin, J., & Sasikumar, R. (2025). Advances in DevSecOps and the future of cybersecurity using automation.

Romeo, J., Raglianti, M., Nagy, C., & Lanza, M. (2024). Capturing and understanding the drift between design, implementation, and documentation. In *Proceedings of the IEEE/ACM International Conference on Program Comprehension*.

Rukmono, S. A., Zamfir, F., Ochoa, L., & Chaudron, M. R. V. (2021). From expert to novice: An empirical study on software architecture explanations. In *Proceedings of the IEEE/ACM International Conference on Program Comprehension*.

Saraiva, D., Gameleira Neto, J., Kulesza, U., Freitas, G., Reboucas, R., & Coelho, R. (2021). Technical debt tools: A systematic mapping study. In *Proceedings of the International Conference on Enterprise Information Systems*.

TehraniJamsaz, A., Chen, H., & Jannesari, A. (2023). GraphBinMatch: Graph-based similarity learning for cross-language binary and source code matching. *arXiv*. https://arxiv.org/abs/2304.04658

Triola, M. F. (2022). *Elementary statistics* (14th ed.). Pearson.

Vaswani, A., et al. (2020). On the computational complexity of self-attention.

Wang, W., Li, G., Ma, B., Xia, X., & Jin, Z. (2021). Detecting code clones with graph neural network and flow-augmented abstract syntax tree.

Yas, Q. M., Alazzawi, A., & Rahmatullah, B. (2023). A comprehensive review of software development life cycle methodologies: Pros, cons, and future directions. Iraqi Journal for Computer Science and Mathematics, 4(4). https://doi.org/10.52866/ijcsm.2023.04.04.014

Yeboah, J., & Popoola, S. (2023). Efficacy of static analysis tools for software defect detection on open-source projects. In *Proceedings of the Annual Conference on Computational Science and Computational Intelligence*.

Zhang, X., et al. (2024). K-ASTRO: Structure-aware adaptation of LLMs for code vulnerability detection.

Zhang, Y., Sandborn, M., Larson, S., Huang, Y., & Leach, K. (2025). K-ASTRO: Structure-aware adaptation of LLMs for code vulnerability detection. In *Proceedings of the ACM Conference*. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Zhang, Z., & Saber, T. (2025). AST-enhanced or AST-overloaded? The surprising impact of hybrid graph representations on code clone detection. *arXiv*. https://arxiv.org/abs/2506.14470

Zhi, X., Tang, B., Zhu, Y., Yan, X., Yin, Z., & Zhou, M. (2023). CoroGraph: Bridging cache efficiency and work efficiency for graph algorithm execution.

Chung, J., Ko, R., Yoo, W., Onizuka, M., Kim, S., Kim, T.-W., & Shin, W.-Y. (2025). *GraphCompliance: Aligning policy and context graphs for LLM-based regulatory compliance*. arXiv. https://arxiv.org/abs/2510.26309

Fan, Y., Xia, X., Lo, D., Hassan, A. E., Wang, Y., & Li, S. (2021). A differential testing approach for evaluating abstract syntax tree mapping algorithms. In *2021 IEEE/ACM*

43rd International Conference on Software Engineering (ICSE). IEEE. https://doi.org/10.1109/ICSE43902.2021.00108

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology, 33*(8), Article 220. https://doi.org/10.1145/3695988

Liu, J., Zeng, J., Wang, X., & Liang, Z. (2023). Learning graph-based code representations for source-level functional similarity detection. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE. https://doi.org/10.1109/ICSE48619.2023.00040

Nesteruk, D. (2022). *Design patterns in modern C++20: Reusable approaches for object-oriented software design*. Apress. https://doi.org/10.1007/978-1-4842-7295-4

Nguyen Quang Do, L., Wright, J. R., & Ali, K. (2020). Why do software developers use static analysis tools? A user-centered study of developer needs and motivations. *IEEE Transactions on Software Engineering*.

Park, J., Lee, H., & Ryu, S. (2021). A survey of parametric static analysis. *ACM Computing Surveys, 54*(7), Article 149. https://doi.org/10.1145/3464457

Romeo, J., Raglianti, M., Nagy, C., & Lanza, M. (2024). Capturing and understanding the drift between design, implementation, and documentation. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC)*, 382–386. https://doi.org/10.1145/3643916.3644399

Rukmono, S. A., Zamfirov, F., Ochoa, L., & Chaudron, M. (2025). *From expert to novice: An empirical study on software architecture explanations*. arXiv. https://arxiv.org/abs/2503.08628

TehraniJamsaz, A., Chen, H., & Jannesari, A. (2023). *GraphBinMatch: Graph-based similarity learning for cross-language binary and source code matching*. arXiv. https://arxiv.org/abs/2304.04658

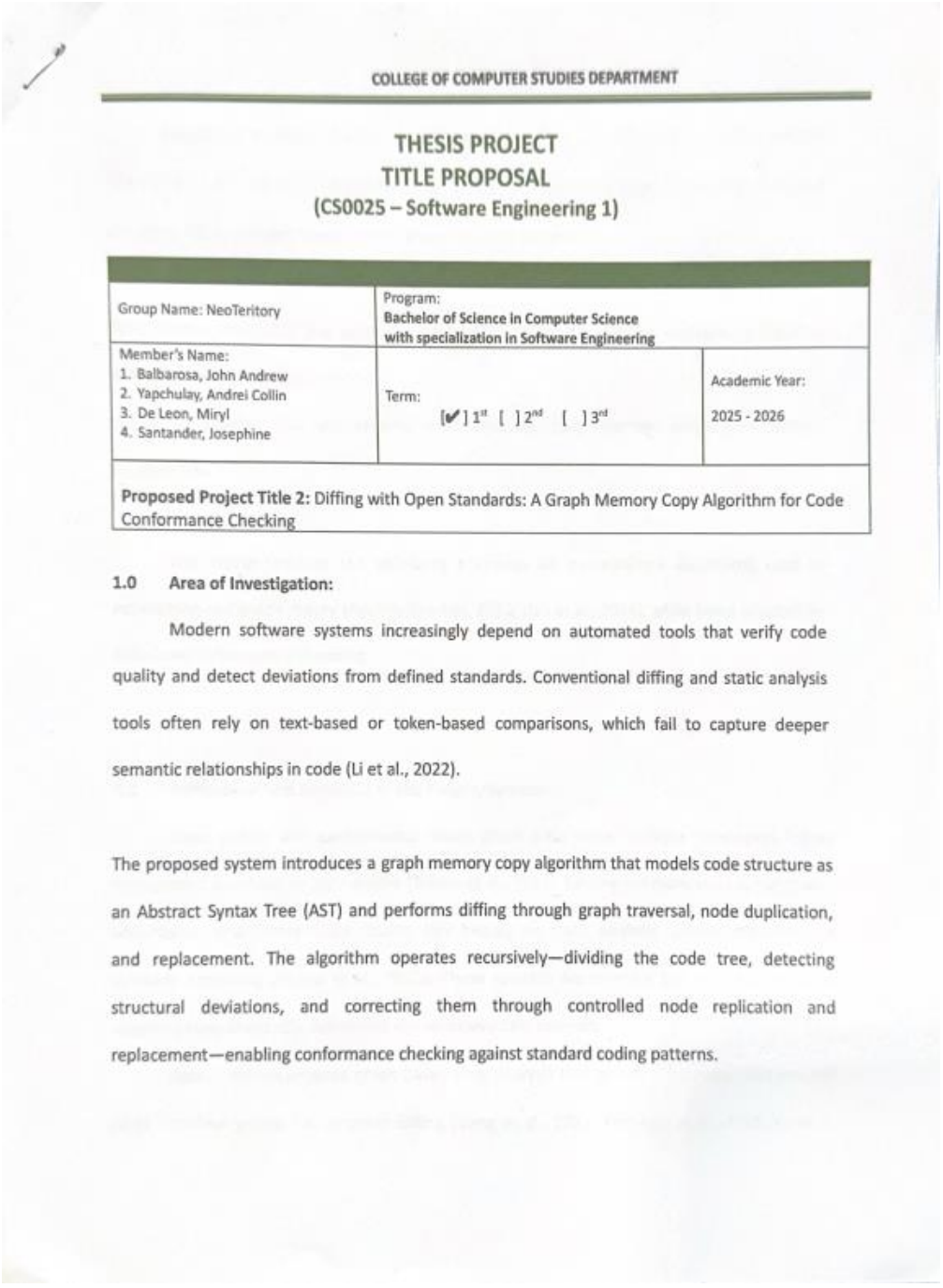
Wang, W., Li, G., Ma, B., Xia, X., & Jin, Z. (2020). *Detecting code clones with graph neural network and flow-augmented abstract syntax tree*. arXiv. https://arxiv.org/abs/2002.08653

Zhang, Z., & Saber, T. (2025). *AST-enhanced or AST-overloaded? The surprising impact of hybrid graph representations on code clone detection*. arXiv. https://arxiv.org/abs/2506.14470

APPENDICES

APPENDIX A

Thesis Title Approval Form



COLLEGE OF COMPUTER STUDIES DEPARTMENT

THESIS PROJECT
TITLE PROPOSAL
(CS0025 – Software Engineering 1)

Group Name: NeoTeritory	Program: **Bachelor of Science in Computer Science with specialization in Software Engineering**	
Member's Name: 1. Balbarosa, John Andrew 2. Yapchulay, Andrei Collin 3. De Leon, Miryl 4. Santander, Josephine	Term: [✔] 1st [ ] 2nd [ ] 3rd	Academic Year: 2025 - 2026
Proposed Project Title 2: Diffing with Open Standards: A Graph Memory Copy Algorithm for Code Conformance Checking		

1.0 Area of Investigation:

Modern software systems increasingly depend on automated tools that verify code quality and detect deviations from defined standards. Conventional diffing and static analysis tools often rely on text-based or token-based comparisons, which fail to capture deeper semantic relationships in code (Li et al., 2022).

The proposed system introduces a graph memory copy algorithm that models code structure as an Abstract Syntax Tree (AST) and performs diffing through graph traversal, node duplication, and replacement. The algorithm operates recursively—dividing the code tree, detecting structural deviations, and correcting them through controlled node replication and replacement—enabling conformance checking against standard coding patterns.

Inspired by localized decision models used in routing and lightweight learning systems (Abdi et al., 2022; He et al., 2024; Nie et al., 2024), this approach emphasizes time and space efficiency. Each node processes only its direct syntactic neighbors:

Time Complexity: O(d) per node — updating exponential moving averages, comparing sub-nodes, and applying correction.

Space Complexity: O(d) per module — maintaining only neighbor nodes and pattern probabilities.

This design mirrors the efficiency principles of decentralized algorithms used in networking and graph theory (Aguirre Sanchez, 2025; Han et al., 2024), while being adapted for code-level conformance checking.

2.0 Background and Rationale of the Project/System:

Code quality and conformance issues often arise when multiple developers follow inconsistent standards or style guides (Tufano et al., 2021). Existing solutions such as Polyspace, SonarQube, and Sentry Code Quality rely heavily on static analysis, pattern matching, or symbolic reasoning (Huang et al., 2023). These systems are effective but resource-intensive, requiring centralized rule definitions and extensive CPU memory.

Recent studies propose graph-based code analysis that transforms source code into AST or control-flow graphs for semantic diffing (Dang et al., 2022; Kechagia et al., 2023; Xu et al.,

2024). However, many such methods still struggle with scalability and adaptability across languages.

To address these challenges, this study adapts Limited-Memory Markov Decision Process (LM-MDP) principles—previously successful in decentralized routing (Abdi et al., 2022; Xiao et al., 2024)—to the software engineering domain. By incorporating short-term memory, node centrality, and probabilistic replacement, the proposed algorithm identifies "faulty nodes" (non-conforming code segments), replicates corrected versions, and reinserts them efficiently.

In essence, this project bridges graph-based diffing with lightweight learning and network optimization theory, creating a model capable of enforcing open coding standards automatically, improving both maintainability and developer productivity.

3.0 Statement of Objectives:

General Objective

To design and validate a graph-based diffing algorithm that automatically detects and corrects non-conforming code structures using localized node analysis and limited memory, achieving high conformance accuracy with minimal computational cost.

Specific Objectives:

1. Use the Model code as an AST graph and develop a graph memory copy algorithm that detects, duplicates, and replaces nodes violating conformance rules.

2. Implement the algorithm and baseline diffing methods (e.g., textual diff, AST-Diff, and ML-based detectors) in Python or MATLAB.
3. Evaluate algorithm performance using metrics such as precision/recall for conformance detection, memory footprint, and processing time across projects of different sizes.
4. Compare results with existing systems such as Polyspace and Nette Coding Standard, demonstrating ≥ 20% efficiency improvement in detecting inconsistent or faulty code segments.
5. Document algorithmic complexity, comparative analysis, and reproducible experiment scripts.

4.0 Scope and Limitations of the Study:

Scope

The study focuses on simulated environments using open-source repositories to test conformance across multiple coding styles (Python, PHP, C++).

It applies graph-based diffing through limited-memory traversal and evaluates both code structure and semantic similarity.

The algorithm can be applied to:

- Code review tools and CI/CD pipelines for automatic correction.
- Educational code analysis systems detecting student syntax deviations.
- Open-standard frameworks like PEP 8 or PSR-12 to maintain consistent syntax and logic flow.

Limitations

No live integration with commercial IDEs or real-time collaborative platforms is included. Performance tests depend on repository complexity and parser accuracy. Full integration with compiler-level feedback or IDE autocompletion systems is out of scope.

5.0 Target Beneficiaries:

1. Developers and QA engineers seeking automatic, language-agnostic conformance tools.
2. Software organizations enforcing open coding standards across teams.
3. Educators and researchers studying program analysis and code evolution.
4. Open-source communities aiming to maintain consistent, high-quality contributions.

6.0 SDG Category Number:

SDG 9 — Industry, Innovation, and Infrastructure. The work advances resilient digital infrastructure by improving the efficiency and reliability of computer networks. (Secondary linkage: SDG 11/13 via indirect efficiency/energy benefits in digital systems.)

7.0 References:

Abdi, F., Ghiasvand, S., & Tofighy, S. (2022). LA-MDPF: A forwarding strategy based on learning automata and Markov decision process in named data networking. Future Generation Computer Systems, 132, 284–297. https://doi.org/10.1016/j.future.2022.01.023

Abdi, F., Ghiasvand, S., & Tofighy, S. (2023). AM-IF: Adaptive multi-path interest forwarding in named data networking. Future Generation Computer Systems, 139, 295–308. https://doi.org/10.1016/j.future.2023.03.021

Aguirre Sanchez, L. P. (2025). MDQ: A QoS-Congestion Aware DRL approach for multi-path routing in SDN. Journal of Network and Computer Applications. https://doi.org/10.1016/j.jnca.2024.103812

Dang, Y., Zhang, Y., & Ren, Z. (2022). Graph-based semantic code diffing for vulnerability detection. IEEE Access, 10, 98714–98728. https://doi.org/10.1109/ACCESS.2022.3209871

Han, H., Tang, J., & Jing, Z. (2024). Wireless sensor network routing optimization based on improved ant colony algorithm in the Internet of Things. Heliyon, 10(1), e23577. https://doi.org/10.1016/j.heliyon.2024.e23577

He, Y., Xiao, G., Zhu, J., Zou, T., & Liang, Y. (2024). Reinforcement learning-based SDN routing empowered by causality detection and GNN. Frontiers in Computational Neuroscience, 18, 1393025. https://doi.org/10.3389/fncom.2024.1393025

Huang, H., Liu, W., & Jiang, Y. (2023). Deep code conformance analysis using transformer-based representations. Information and Software Technology, 161, 107207. https://doi.org/10.1016/j.infsof.2023.107207

Kechagia, M., Mitropoulou, V., & Spinellis, D. (2023). Empirical evaluation of AST-based differencing algorithms in software repositories. Empirical Software Engineering, 28(4), 79. https://doi.org/10.1007/s10664-023-10284-x

Li, S., Zhao, P., & Huang, Z. (2022). ASTDiff++: Efficient abstract syntax tree differencing for large-scale software repositories. Journal of Systems and Software, 194, 111538. https://doi.org/10.1016/j.jss.2022.111538

Nie, W., Chen, Y., Wang, Y., & Ning, L. (2024). Routing technology based on improved ant colony algorithm in space-air-ground integrated networks. EURASIP Journal on Advances in Signal Processing, 2024(34). https://doi.org/10.1186/s13634-024-01131-5

Tufano, M., Watson, C., Penta, M. D., & White, M. (2021). Learning how to automatically enforce coding standards. IEEE Transactions on Software Engineering, 47(12), 2723–2738. https://doi.org/10.1109/TSE.2020.2998431

Xiao, Y., Liu, J., Li, G., Wang, L., & Yang, B. (2024). Scalable QoS-aware multipath routing in hybrid knowledge-defined networking with multiagent deep RL. IEEE Transactions on Mobile Computing, 23(11), 10628–10646. https://doi.org/10.1109/TMC.2024.10475545

Xu, Y., Dong, H., & Feng, Q. (2024). Code conformance validation through graph neural representation learning. ACM Transactions on Software Engineering and Methodology, 33(2), 1–25. https://doi.org/10.1145/3634211

Zhang, J., Ye, M., Guo, Z., & Chao, H. J. (2020). CFR-RL: Traffic engineering with reinforcement learning in SDN. IEEE Journal on Selected Areas in Communications, 38(10), 2249–2259. https://doi.org/10.1109/JSAC.2020.3005841

Approved by:

Fanny C. Abamira
Panel Member 1

11/27 2025
Dr. Hazel Jane L. Pablano
Panel Member 2

Noted by:

11/27 2025
Dr. Hazel Jane L. Pablano
Thesis 1 Course Adviser